VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

Aziz Nagar Gate, C.B. Post, Hyderabad-75

Department of

CSE(DS)

Year : IV B.Tech

Semester : I

BIGDATA ANALYTICS LAB

LAB MANUAL

Course Outcomes

At the end of this course, the student would be able to:

1. To introduce the tools required to manage and analyze big data like Hadoop, NoSql
2. To impart knowledge of map reduce paradigm to solve complex problems Map-Reduce
3. To introduce several new algorithms for big data mining like classification, clustering and finding frequent patterns.

INDEX

Week 1,2:

1. Implement the following Data structures in Java
   a) Linked Lists b) Stacks c) Queues d) Set e) Map

Collections in Java

The **Collection in Java** is a framework that provides an architecture to store and manipulate the group of objects.

All the operations that you perform on a data such as searching, sorting, insertion, manipulation, deletion, etc. can be achieved by Java Collections.

Java Collection means a single unit of objects. Java Collection framework provides many interfaces (Set, List, Queue, Deque, etc.) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet, etc.).

What is Collection in java

A Collection represents a single unit of objects, i.e., a group.

What is a framework in Java

- It provides readymade architecture.
- It represents a set of classes and interface.
- It is optional.

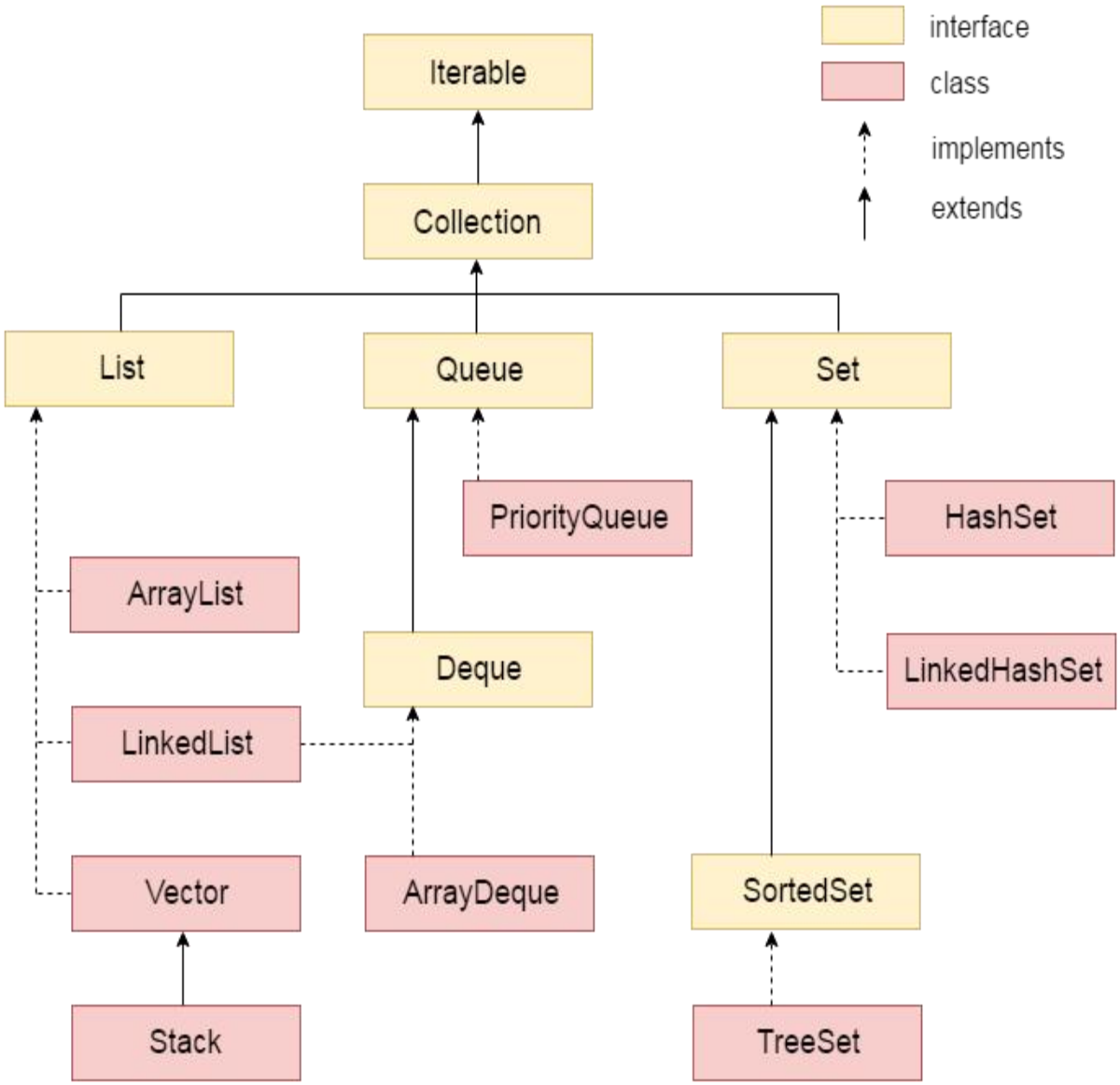
What is Collection framework

The Collection framework represents a unified architecture for storing and manipulating a group of objects. It has:

1. Interfaces and its implementations, i.e., classes
2. Algorithm

Hierarchy of Collection Framework

Let us see the hierarchy of Collection framework. The **java.util** package contains all the classes and interfaces for Collection framework.

1. 

1.LinkedList: Methods of Java LinkedList

Method	Description
void add(int index, Object element)	It is used to insert the specified element at the specified position index in a list.
void addFirst(Object o)	It is used to insert the given element at the beginning of a list.
void addLast(Object o)	It is used to append the given element to the end of

	a list.
int size()	It is used to return the number of elements in a list
boolean add(Object o)	It is used to append the specified element to the end of a list.
boolean contains(Object o)	It is used to return true if the list contains a specified element.
boolean remove(Object o)	It is used to remove the first occurence of the specified element in a list.
Object getFirst()	It is used to return the first element in a list.
Object getLast()	It is used to return the last element in a list.
int indexOf(Object o)	It is used to return the index in a list of the first occurrence of the specified element, or -1 if the list does not contain any element.
int lastIndexOf(Object o)	It is used to return the index in a list of the last occurrence of the specified element, or -1 if the list does not contain any element.

a) Linked Lists:

```
import java.util.*;

public class Test
{
   public static void main(String args[])
   {
      // Creating object of class linked list
      LinkedList<String> object = new LinkedList<String>();

      // Adding elements to the linked list
      object.add("A");
      object.add("B");
      object.addLast("C");
      object.addFirst("D");
      object.add(2, "E");
      object.add("F");
      object.add("G");
      System.out.println("Linked list : " + object);

      // Removing elements from the linked list
      object.remove("B");
      object.remove(3);
      object.removeFirst();
      object.removeLast();
      System.out.println("Linked list after deletion: " + object);

      // Finding elements in the linked list
      boolean status = object.contains("E");

      if(status)
         System.out.println("List contains the element 'E' ");
      else
         System.out.println("List doesn't contain the element 'E'");

      // Number of elements in the linked list
      int size = object.size();
      System.out.println("Size of linked list = " + size);

      // Get and set elements from linked list
      Object element = object.get(2);
      System.out.println("Element returned by get() : " + element);
      object.set(2, "Y");
      System.out.println("Linked list after change : " + object);
```

```
    }
}
```

Run on IDE

Output:

```
Linked list : [D, A, E, B, C, F, G]
Linked list after deletion: [A, E, F]
List contains the element 'E'
Size of linked list = 3
Element returned by get() : F
Linked list after change : [A, E, Y]
```

B) Stack

```
import java.io.*;
import java.util.*;

class Test
{
    // Pushing element on the top of the stack
    static void stack_push(Stack<Integer> stack)
    {
        for(int i = 0; i < 5; i++)
        {
            stack.push(i);
        }
    }

    // Popping element from the top of the stack
    static void stack_pop(Stack<Integer> stack)
    {
        System.out.println("Pop :");

        for(int i = 0; i < 5; i++)
        {
            Integer y = (Integer) stack.pop();
            System.out.println(y);
        }
    }

    // Displaying element on the top of the stack
    static void stack_peek(Stack<Integer> stack)
```

```
    {
       Integer element = (Integer) stack.peek();
       System.out.println("Element on stack top : " + element);
    }
      // Searching element in the stack
    static void stack_search(Stack<Integer> stack, int element)
    {
       Integer pos = (Integer) stack.search(element);

       if(pos == -1)
          System.out.println("Element not found");
       else
          System.out.println("Element is found at position " + pos);
    }

     public static void main (String[] args)
    {
       Stack<Integer> stack = new Stack<Integer>();

       stack_push(stack);
       stack_pop(stack);
       stack_push(stack);
       stack_peek(stack);
       stack_search(stack, 2);
       stack_search(stack, 6);
    }
}
```

```
Pop :
4
3
2
1
0
Element on stack top : 4
Element is found at position 3
Element not found
```

C) QUEUES:

// Java orogram to demonstrate working of Queue

```
// interface in Java
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample
{
  public static void main(String[] args)
  {
    Queue<Integer> q = new LinkedList<>();

    // Adds elements {0, 1, 2, 3, 4} to queue
    for (int i=0; i<5; i++)
     q.add(i);

    // Display contents of the queue.
    System.out.println("Elements of queue-"+q);

    // To remove the head of queue.
    int removedele = q.remove();
    System.out.println("removed element-" + removedele);

    System.out.println(q);

    // To view the head of queue
    int head = q.peek();
    System.out.println("head of queue-" + head);

    // Rest all methods of collection interface,
    // Like size and contains can be used with this
    // implementation.
    int size = q.size();
    System.out.println("Size of queue-" + size);
  }
}
```

Run on IDE

Output:

```
Elements of queue-[0, 1, 2, 3, 4]
```

```
removed element-0
[1, 2, 3, 4]
head of queue-1
Size of queue-4
```

D) SET:

Java Set is a part of java.util package and extends java.util.Collection interface. It does not allow the use of duplicate elements and at max can accommodate only one null element. Few important features of Java Set interface are as follows:

- The set interface is an unordered collection of objects in which duplicate values cannot be stored.
- The Java Set does not provide control over the position of insertion or deletion of elements.
- Basically, Set is implemented by HashSet, LinkedHashSet or TreeSet (sorted representation).
- Set has various methods to add, remove clear, size, etc to enhance the usage of this interface

```
import java.util.*;
public class Set_example
{
   public static void main(String[] args)
   {
      // Set demonstration using HashSet
      Set<String> hash_Set = new HashSet<String>();
      hash_Set.add("Geeks");
      hash_Set.add("For");
      hash_Set.add("Geeks");
      hash_Set.add("Example");
      hash_Set.add("Set");
      System.out.print("Set output without the duplicates");

      System.out.println(hash_Set);

      // Set demonstration using TreeSet
      System.out.print("Sorted Set after passing into TreeSet");
      Set<String> tree_Set = new TreeSet<String>(hash_Set);
      System.out.println(tree_Set);
   }
```

```
    }
```

Run on IDE

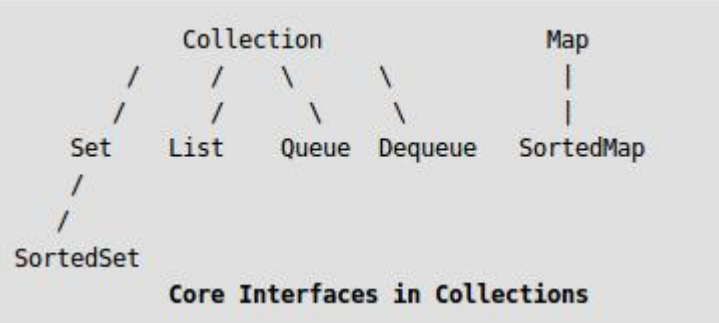
Output:

```
Set output without the duplicates[Set, Example, Geeks, For]
Sorted Set after passing into TreeSet[Example, For, Geeks, Set]
```

E.MAP

Map Interface in Java

The java.util.Map interface represents a mapping between a key and a value. The Map interface is not a subtype of the Collection interface. Therefore it behaves a bit different from the rest of the collection types.



Few characteristics of the Map Interface are:

1. A Map cannot contain duplicate keys and each key can map to at most one value. Some implementations allow null key and null value like the HashMap and LinkedHashMap, but some do not like the TreeMap.
2. The order of a map depends on specific implementations, e.g TreeMap and LinkedHashMap have predictable order, while HashMap does not.
3. There are two interfaces for implementing Map in java: Map and SortedMap, and three classes: HashMap, TreeMap and LinkedHashMap.

Why and When to use Maps?

Maps are perfect to use for key-value association mapping such as dictionaries. The maps are used to perform lookups by keys or when someone wants to retrieve and update elements by keys. Some examples are:

- A map of error codes and their descriptions.
- A map of zip codes and cities.
- A map of managers and employees. Each manager (key) is associated with a list of employees (value) he manages.
- A map of classes and students. Each class (key) is associated with a list of students (value).

Hash Map

```
/* Java program to print frequencies of all elements using
   HashMap */

import java.util.*;

  class Main

{

    // This function prints frequencies of all elements

    static void printFreq(int arr[])

    {

        // Creates an empty HashMap

        HashMap<Integer, Integer> hmap =

                    new HashMap<Integer, Integer>();

          // Traverse through the given array

        for (int i = 0; i < arr.length; i++)

        {

            Integer c = hmap.get(arr[i]);

              // If this is first occurrence of element

            if (hmap.get(arr[i]) == null)

               hmap.put(arr[i], 1);
```

```
            // If elements already exists in hash map
          else
            hmap.put(arr[i], ++c);
       }
        // Print result
       for (Map.Entry m:hmap.entrySet())
         System.out.println("Frequency of " + m.getKey() +
                            " is " + m.getValue());
   }
    // Driver method to test above method
   public static void main (String[] args)
   {
      int arr[] = {10, 34, 5, 10, 3, 5, 10};
      printFreq(arr);
   }
}
```

Output:

```
Frequency of 34 is 1
Frequency of 3 is 1
Frequency of 5 is 2
Frequency of 10 is 3
```

Tree Map

```
/* Java program to print frequencies of all elements using
   TreeMap */
import java.util.*;

class Main
{
    // This function prints frequencies of all elements
    static void printFreq(int arr[])
    {
        // Creates an empty TreeMap
        TreeMap<Integer, Integer> tmap =
                    new TreeMap<Integer, Integer>();

        // Traverse through the given array
        for (int i = 0; i < arr.length; i++)
        {
            Integer c = tmap.get(arr[i]);

            // If this is first occurrence of element
            if (tmap.get(arr[i]) == null)
               tmap.put(arr[i], 1);

            // If elements already exists in hash map
            else
              tmap.put(arr[i], ++c);
        }

        // Print result
        for (Map.Entry m:tmap.entrySet())
          System.out.println("Frequency of " + m.getKey() +
                           " is " + m.getValue());
    }

    // Driver method to test above method
    public static void main (String[] args)
    {
        int arr[] = {10, 34, 5, 10, 3, 5, 10};
        printFreq(arr);
    }
}
```

Run on IDE

Output:

```
Frequency of 3 is 1
Frequency of 5 is 2
Frequency of 10 is 3
Frequency of 34 is 1
```

Week 3: Perform setting up and Installing Hadoop in

- Pseudo distributed mode

After going through the fundamentals of Hadoop, HDFS & MapReduce, its time we move on to install Hadoop on your system. Before the actual installation, it is important to understand the modes in which Hadoop can be run.

There are three modes in which Hadoop can be installed and run. They are:-

1. Standalone Mode
2. Pseudo Distributed Mode
3. Fully Distributed Mode

Pseudo Distributed Mode

This mode helps you mimic a multi-server installation on a single machine. Pseudo Distributed mode also runs on a single machine, but it has all the daemons running in a separate process. Also it will have the files stored on HDFS and not on the local machine. Thus in be seen as small scale implementation before you run on an actual cluster with 1000s of nodes.

Development Tip – So before you run your code on a real Hadoop cluster, following a few steps can make your life easy with MapReduce.

1. Always Unit Test your code first.
2. Start with a very small portion of the input.
3. Test locally (in LocalJobRunner mode) with the small input.
4. Test in Pseudo Distributed mode with daemons running (to view the performance of your code with the small input).
5. Finally if you are satisfied with running you code and its performance, you are ready to run it in Fully Distributed Mode on a real cluster.

Hadoop software can be installed in three modes of operation:

Pseudo Distributed Mode: In this mode also, Hadoop software is installed on a Single Node. Various daemons of Hadoop will run on the same machine as separate java processes. Hence all the daemons namely NameNode, DataNode, SecondaryNameNode, JobTracker, TaskTracker run on single machine.

Hadoop Installation: Psuedo Distributed Mode(Locally)

Steps for Installation

1. Edit the file /home/Hadoop_dev/hadoop2/etc/hadoop/core-site.xml as below:

```
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

Note: This change sets the namenode ip and port.

2. Edit the file /home/Hadoop_dev/hadoop2/etc/hadoop/hdfs-site.xml as below:

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
</configuration>
```

Note: This change sets the default replication count for blocks used by HDFS.

3. We need to setup password less login so that the master will be able to do a password-less ssh to start the daemons on all the slaves.

Check if ssh server is running on your host or not:

a. ssh localhost (enter your password and if you are able to login then ssh server is running)

b. In step a. if you are unable to login, then install ssh as follows:

```
sudo apt-get install ssh
```

c. Setup password less login as below:

i. ssh-keygen -t dsa -P '' -f ~/.ssh/id_dsa

ii. cat ~/.ssh/id_dsa.pub >> ~/.ssh/authorized_keys

4. We can run Hadoop jobs locally or on YARN in this mode. In this Post, we will focus on running the jobs **locally.**

5. Format the file system. When we format namenode it formats the meta-data related to data-nodes. By doing that, all the information on the datanodes are lost and they can be reused for new data:

a. bin/hdfs namenode –format

6. Start the daemons

a. sbin/start-dfs.sh (Starts NameNode and DataNode)

You can check If NameNode has started successfully or not by using the following web interface: http://0.0.0.0:50070 . If you are unable to see this, try to check the logs in the /home/hadoop_dev/hadoop2/logs folder.

7. You can check whether the daemons are running or not by issuing Jps command.

8. This finishes the installation of Hadoop in pseudo distributed mode.

9. Let us run the same example we can in the previous blog post:

i) Create a new directory on the hdfs

bin/hdfs dfs -mkdir –p /user/hadoop_dev

ii) Copy the input files for the program to hdfs:

bin/hdfs dfs -put etc/hadoop input

iii) Run the program:

bin/hadoop jar share/hadoop/mapreduce/hadoop-mapreduce-examples-2.6.0.jar grep input output 'dfs[a-z.]+'

iv) View the output on hdfs:

bin/hdfs dfs -cat output/*

10. Stop the daemons when you are done executing the jobs, with the below command:

sbin/stop-dfs.sh

Hadoop Installation – Psuedo Distributed Mode(YARN)

Steps for Installation

1. Edit the file /home/hadoop_dev/hadoop2/etc/hadoop/mapred-site.xml as below:

```
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
```

2. Edit the fie /home/hadoop_dev/hadoop2/etc/hadoop/yarn-site.xml as below:

```
<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>
```

Note: This particular configuration tells MapReduce how to do its shuffle. In this case it uses the mapreduce_shuffle.

3. Format the NameNode:

```
bin/hdfs namenode –format
```

4. Start the daemons using the command:

```
sbin/start-yarn.sh
```

This starts the daemons ResourceManager and NodeManager.

Once this command is run, you can check if ResourceManager is running or not by visiting the following URL on browser : http://0.0.0.0:8088 . If you are unable to see this, check for the logs in the directory: /home/hadoop_dev/hadoop2/logs

5. To check whether the services are running, issue a jps command. The following shows all the services necessary to run YARN on a single server:

```
$ jps
15933 Jps
```

15567 ResourceManager

15785 NodeManager

6. Let us run the same example as we ran before:

i) Create a new directory on the hdfs

```
bin/hdfs dfs -mkdir –p /user/hadoop_dev
```

ii) Copy the input files for the program to hdfs:

```
bin/hdfs dfs -put etc/hadoop input
```

iii) Run the program:

```
bin/yarn jar share/hadoop/mapreduce/hadoop-mapreduce-examples-2.6.0.jar grep input output 'dfs[a-z.]+'
```

iv) View the output on hdfs:

```
bin/hdfs dfs -cat output/*
```

7. Stop the daemons when you are done executing the jobs, with the below command:

```
sbin/stop-yarn.sh
```

Week 4:

3. Implement the following file management tasks in Hadoop:
    - Adding files and directories
    - Retrieving files
    - Deleting files

    Hint: A typical Hadoop workflow creates data files (such as log files) elsewhere and copies them into HDFS using one of the above command line utilities.

HDFS File System Commands:

Apache Hadoop has come up with a simple and yet basic Command Line interface, a simple interface to access the underlying Hadoop Distributed File System. In this section we will introduce you to the basic and the most useful HDFS File System Commands which will be more or the like similar to UNIX file system commands. Once the Hadoop daemons are started and UP and Running, HDFS file system is ready to be used. The file system operations like creating directories, moving files, adding files, deleting files, reading files and listing directories can be done seamlessly on the same.

We can get list of FS Shell commands with below command.

$ hadoop fs -help

user@ubuntu1:~$ hadoop fs -help

Usage: hadoop fs [generic options]

[-appendToFile ...]

[-cat [-ignoreCrc] ...]

[-checksum ...]

[-chgrp [-R] GROUP PATH...]

[-chmod [-R] PATH...]

[-chown [-R] [OWNER][:[GROUP]] PATH...]

[-copyFromLocal [-f] [-p] ...]

[-copyToLocal [-p] [-ignoreCrc] [-crc] ...]

[-count [-q] ...]

[-cp [-f] [-p | -p[topax]] ...]

[-createSnapshot []]

[-deleteSnapshot]

[-df [-h] [...]]

[-du [-s] [-h] ...]

[-expunge]

[-get [-p] [-ignoreCrc] [-crc] ...]

[-getfacl [-R]]

[-getfattr [-R] {-n name | -d} [-e en]]

[-getmerge [-nl]]

[-help [cmd ...]]

[-ls [-d] [-h] [-R] [...]]

[-mkdir [-p] ...]

[-moveFromLocal ...]

[-moveToLocal]

[-mv ...]

[-put [-f] [-p] ...]

[-renameSnapshot]

[-rm [-f] [-r|-R] [-skipTrash] ...]

[-rmdir [--ignore-fail-on-non-empty]

...]

[-setfacl [-R] [{-b|-k} {-m|-x }]|[--set]]

[-setfattr {-n name [-v value] | -x name}]

[-setrep [-R] [-w] ...]

[-stat [format] ...]

[-tail [-f]]

[-test -[defsz]]

[-text [-ignoreCrc] ...]

[-touchz ...]

[-usage [cmd ...]]

Most of the commands that we use on a HDFS environment are listed as above, from this thorough list of commands we will take a look at the some of the most important commands with examples. Let us take a look into the commands and its usage then:

1. mkdir:

This is no different from the UNIX **mkdir** command and is used to create a directory on a HDFS environment. The option –**p** is used to mention not to fail if the directory already exists.

Syntax:

$ hadoop fs -mkdir [-p]

Usage:

$ hadoop fs -mkdir /user/hadoop/

$ hadoop fs -mkdir /user/data/

In order to create subdirectories, the parent directory needs to be existing already. If the condition is not met then 'No such file or directory' message will be returned.

2. ls:

This is no different from the UNIX **ls** command and it is used for listing the directories present under a specific directory in an HDFS system. The –**lsr** command may be used for the recursive listing of the directories and files under a specific folder.

The –**d** option is used to list the directories as plain files

The –**h** option is used to format the sizes of files into a human readable manner than just number of bytes

The –**R** option is used to recursively list the contents of directories

Syntax:

$ hadoop fs -ls [-d] [-h] [-R]

Usage:

$ hadoop fs -ls /

$ hadoop fs -lsr /

The command above will match the specified file pattern, directory entries are of the form (as shown below)

permissions - userId groupId sizeOfDirectory(in bytes) modificationDate(yyyy-MM-dd HH:mm) directoryName

3. put:

This command is used to copy files from the local file system to the HDFS filesystem. This is very much similar to the –copyFromLocal command. Copying of the files fails if the file already exists, unless the –**f** flag is given to the command. This overwrites the destination if the file already exists before the copy

The –**p** flag preserves the access, modification time, ownership and the mode

Syntax:

$ hadoop fs -put [-f] [-p] ...

Usage:

$ hadoop fs -put sample.txt /user/data/

4. get:

This command Copies files from HDFS file system to the local file system, just the opposite of the command that we have seen just now. This is similar to the –copyToLocal command

Usage:

$ hadoop fs -get /user/data/sample.txt workspace/

5. cat:

This command is similar to the UNIX **cat** command and is used for displaying the contents of a file on the console.

Usage:

$ hadoop fs -cat /user/data/sampletext.txt

6. cp:

This command is similar to the UNIX **cp** command, and it is used for copying files from one directory to another directory within the HDFS file system.

Usage:

$ hadoop fs -cp /user/data/sample1.txt /user/hadoop1

$ hadoop fs -cp /user/data/sample2.txt /user/test/in1

7. mv:

This command is similar to the UNIX **mv** command, and it is used for moving a file from one directory to another directory within the HDFS file system.

Usage:

$ hadoop fs -mv /user/hadoop/sample1.txt /user/text/

8. rm:

This command is similar to the UNIX **rm** command, and it is used for removing a file from the HDFS file system. The command –rmr can be used to delete files recursively.

Directories can't be deleted by the –**rm** command, we need to use the –**rm r** (recursive remove) command to delete directories and files inside them. Only files can be removed using the –**rm** command.

The –**skipTrash** option is used to bypass the trash, if enabled, and then it immediately deletes the source as mentioned in the tag.

The –**f** option is used to mention that if there is no file existing, then not to display any diagnostic message or even to modify the exit status to reflect on showing up an error.

The –**rR** option is used to recursively delete directories

Syntax:

$ hadoop fs -rm [-f] [-r|-R] [-skipTrash]

Usage:

$ hadoop fs -rm -r /user/test/sample.txt

9. getmerge:

This is the most important and the most useful command on the HDFS filesystem, when trying to read the contents of a MapReduce job or PIG job's output files. This is used for merging a list of files in a directory on the HDFS filesystem into a single local file on the local filesystem.

Usage:

$ hadoop fs -getmerge /user/data

10. setrep:

This command is used to change the replication factor of a file to a specific count instead of the default replication factor for the remaining in the HDFS file system. If is a directory then the command will recursively change the replication factor of all the residing files in the directory tree as per the input provided.

The –**w** option is used to request the command to wait for the replication to be completed. This operation may take considerably longer duration

The –**R** option is used to accept for backward capability and has no effect

Syntax:

$ hadoop fs -setrep [-R] [-w]

Usage:

$ hadoop fs -setrep -R /user/hadoop/

11. touchz:

This command can be used to create a file of zero bytes size in HDFS filesystem.

Usage:

$ hadoop fs -touchz URI

12. test:

This command is used to test a HDFS file's existence of zero length of the file or whether if it is a directory or not.

The –**d** option is used to check whether if it is a directory or not, returns 0 if it is a directory

The –**e** option is used to check whether the exists or not, returns 0 if the exists

The –**f** option is used to check whether the is a file or not, returns 0 if the file exists

The –**s** option is used to check whether the file size is greater than 0 bytes or not, returns 0 if the size is greater than 0 bytes

The –**z** option is used to check whether the file size is zero bytes or not. If the file size is zero bytes, then returns 0 or else returns 1.

Usage:

$ hadoop fs -test -[defsz] /user/test/test.txt

13. expunge:

This command is used to empty the trash available in a HDFS system.

Syntax:

$ hadoop fs –expunge

Usage:

user@ubuntu1:~$ hadoop fs –expunge

17/10/15 10:15:22 INFO fs.TrashPolicyDefault: Namenode trash configuration: Deletion interval = 0 minutes, Emptier interval = 0 minutes.

14. appendToFile:

This command appends the contents of all the given local files to the provided destination file on the HDFS filesystem. The destination file will be created if it is not existing earlier. If the is -, then the input is read from **stdin**.

Syntax:

$ hadoop fs -appendToFile

Usage:

user@ubuntu1:~$ hadoop fs -appendToFile derby.log data.tsv /in/appendfile

user@ubuntu1:~$ hadoop fs -cat /in/appendfile

Sun Oct 15 14:41:10 IST 2017 Thread[main,5,main] Ignored duplicate property derby.module.dataDictionary in jar:file:/home/user/Downloads/apache-hive-0.14.0-bin/lib/hive-jdbc-0.14.0-standalone.jar!/org/apache/derby/modules.properties

Sun Oct 15 14:41:10 IST 2017 Thread[main,5,main] Ignored duplicate property derby.module.lockManagerJ1 in jar:file:/home/user/Downloads/apache-hive-0.14.0-bin/lib/hive-jdbc-0.14.0-standalone.jar!/org/apache/derby/modules.properties

Sun Oct 15 14:41:10 IST 2017 Thread[main,5,main] Ignored duplicate property derby.env.classes.dvfJ2 in jar:file:/home/user/Downloads/apache-hive-0.14.0-bin/lib/hive-jdbc-0.14.0-standalone.jar!/org/apache/derby/modules.properties

15. tail:

This command is used to show the last 1KB of the file.

The –**f** option is used to the show appended data as the file grows

Syntax:

$ hadoop fs -tail [-f]

Usage:

user@tri03ws-386:~$ hadoop fs -tail /in/appendfile

Sun Oct 15 14:41:10 IST 2017:

Booting Derby version The Apache Software Foundation - Apache Derby - 10.10.1.1 - (1458268): instance a816c00e-0149-e638-0064-0000093808b8 on database directory /home/user/metastore_db with class loader sun.misc.Launcher$AppClassLoader@3485def8

Loaded from file:/home/user/Downloads/apache-hive-0.14.0-bin/lib/derby-10.10.1.1.jar

java.vendor=Oracle Corporation

java.runtime.version=1.7.0_65-b32

user.dir=/home/user

os.name=Linux

os.arch=amd64

os.version=3.13.0-39-generic

derby.system.home=null

Database Class Loader started - derby.database.classpath=''

16. stat:

This command is used to print the statistics about the file / directory at in the specified format. Format accepts file size in blocks (%b), group name of the owner (%g) and the file name (%n), block size (%o), replication (%r), user name of the owner (%u), modification date (%y, %Y)

Syntax:

$ hadoop fs -stat [format]

Usage:

$hadoop fs -stat ./cse

user@tri03ws-386:~$ hadoop fs -stat /in/appendfile

2014-11-26 04:57:04

user@tri03ws-386:~$ hadoop fs -stat %Y /in/appendfile

1416977824841

user@tri03ws-386:~$ hadoop fs -stat %b /in/appendfile

20981

user@tri03ws-386:~$ hadoop fs -stat %r /in/appendfile

1

user@tri03ws-386:~$ hadoop fs -stat %o /in/appendfile

134217728

17. setfattr:

This command sets an extended attribute name and value for a file or directory on the HDFS filesystem.

The –**n** option is used to provide the extended attribute name

The –**x** option is used to remove the extended attribute , file or directory

The –**v** option is used to provide the extended attribute value. There are 3 different encoding methods available for the value.

1. If the argument is enclosed in double quotes, then the value is the string inside the quotes

2. If the argument is prefixed with 0x or 0X, then it is taken as a hexadecimal number

3. If the argument begins with 0s or 0S, then it is taken as a Base64 encoding

Syntax:

$ hadoop fs -setfattr {-n name [-v value] | -x name}

18. df:

This command is used to show the capacity, free and the used space available on the HDFS filesystem. If the filesystem has multiple partitions and if there is no path is mentioned to any specific partition, then the status of the root partition will be displayed for us to know.

The –**h** option is used to format the sizes of the files in a human readable manner rather than number of bytes.

Syntax:

$ hadoop fs -df [-h] [...]

19. du:

This command is used to show the amount of space in Bytes that has been used by the files that match the specified file pattern. Even without the –s option, this only shows the size summaries one level deep in the directory.

The –**s** option is used to show the size of each individual file that matches the pattern, shows the total (summary) size

The –**h** option is used to format the sizes of the files in a human readable manner rather than number of bytes.

Syntax:

$ hadoop fs -du [-s] [-h]

20. count:

This command is used to count the number of directories, files and bytes under the path that match the provided file pattern.

Syntax:

$ hadoop fs -count [-q]

The output columns are as follows:

1. DIR_COUNT FILE_COUNT CONTENT_SIZE FILE_NAME

2. QUOTA REMAINING_QUOTA SPACE_QUOTA REMAINING_SPACE_QUOTA

3. DIR_COUNT FILE_COUNT CONTENT_SIZE FILE_NAME

21. chgrp:

This command is used to change the group of a file or a path.

Syntax:

$ hadoop fs -chgrp [-R] groupname

22. chmod:

This command is used to change the permissions of a file, this command works as similar to that of LINUX's shell command **chmod** with a few exceptions.

The –**R** option is used to modify the files recursively and it is the only option that is being supported currently.

The is the same as the mode used for the shell command. The letters that are recognized are '**rwxXt**'.

The is the mode specified in 3 or 4 digits. The first may be 0 or 1 to turn the sticky bit OFF or ON respectively. Unlike the shell command, it is not at all possible to specify only part of the mode.

Syntax:

$ hadoop fs -chmod [-R] PATH

23. chown:

This command is used to change the owner and group of a file. This command is similar to the shell's **chown** command with a few exceptions.

If only the owner of the group is specified then only the owner of the group is modified via this command. The owner and group names may only consists of digits, alphabets and any of the characters mentioned here [-_./@a-zA-Z0-9]. The names thus specified are case sensitive as well.

It is better to avoid using '.' to separate user name and the group just the way LINUX allows it. If the user names have dots in them and if you are using local file system, you might see surprising results since the shell command chown is used for the local file alone.

The –**R** option modifies the files recursively and is the only option that is being supported currently

Syntax:

$ hadoop fs -chown [-R] [OWNER][:[GROUP]] PATH

Week 5: Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm.

In Hadoop, MapReduce is a computation that decomposes large manipulation jobs into individual tasks that can be executed in parallel cross a cluster of servers. The results of tasks can be joined together to compute final results.

MapReduce consists of 2 steps:

Map Function – It takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (Key-Value pair).

Example – (Map function in Word Count)

Input	Set of data	Bus, Car, bus, car, train, car, bus, car, train, bus, TRAIN,BUS, buS, caR, CAR, car, BUS, TRAIN

Output	Convert into another set of data (Key,Value)	(Bus,1), (Car,1), (bus,1), (car,1), (train,1), (car,1), (bus,1), (car,1), (train,1), (bus,1), (TRAIN,1),(BUS,1), (buS,1), (caR,1), (CAR,1), (car,1), (BUS,1), (TRAIN,1)

Reduce Function – Takes the output from Map as an input and combines those data tuples into a smaller set of tuples.

Example – (Reduce function in Word Count)

Input **(output of Map function)**	Set of Tuples	(Bus,1), (Car,1), (bus,1), (car,1), (train,1), (car,1), (bus,1), (car,1), (train,1), (bus,1), (TRAIN,1),(BUS,1), (buS,1), (caR,1), (CAR,1), (car,1), (BUS,1), (TRAIN,1)
Output	Converts into smaller set of tuples	(BUS,7), (CAR,7), (TRAIN,4)

Work Flow of Program

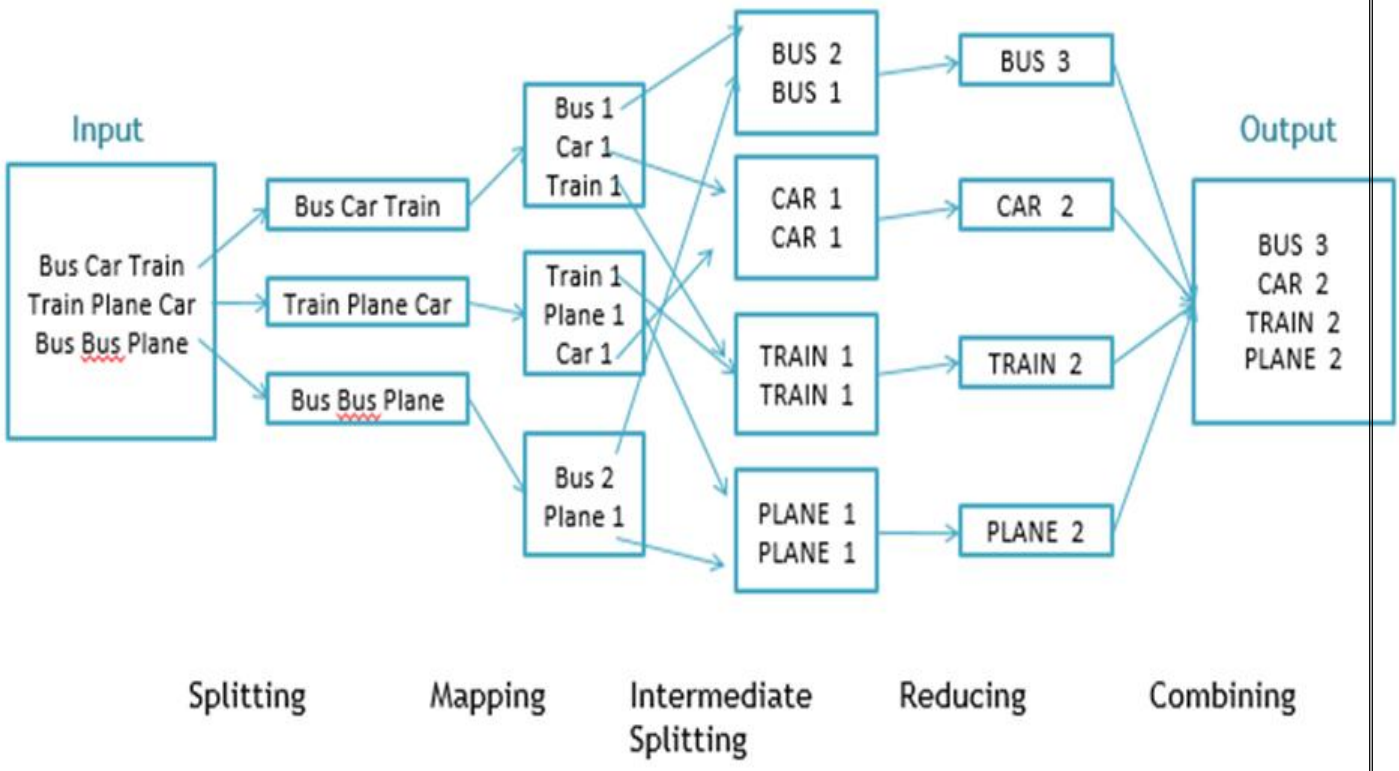


Fig. WorkFlow of MapReducing

Workflow of MapReduce consists of 5 steps

1. **Splitting** – The splitting parameter can be anything, e.g. splitting by space, comma, semicolon, or even by a new line ('\n').
2. **Mapping** – as explained above
3. **Intermediate splitting** – the entire process in parallel on different clusters. In order to group them in "Reduce Phase" the similar KEY data should be on same cluster.
4. **Reduce** – it is nothing but mostly group by phase
5. **Combining** – The last phase where all the data (individual result set from each cluster) is combine together to form a Result

Now Let's See the Word Count Program in Java

Fortunately we don't have to write all of the above steps, we only need to write the splitting parameter, Map function logic, and Reduce function logic. The rest of the remaining steps will execute automatically.

Make sure that Hadoop is installed on your system with java idk

Steps

Step 1. Open Eclipse> File > New > Java Project >(Name it – MRProgramsDemo) > Finish

Step 2. Right Click > New > Package (Name it - PackageDemo) > Finish

Step 3. Right Click on Package > New > Class (Name it - WordCount)

Step 4. Add Following Reference Libraries –

Right Click on Project > Build Path> Add External Archivals

- */usr/lib/hadoop-0.20/***hadoop-core.jar**
- *Usr/lib/hadoop-0.20/lib/***Commons-cli-1.2.jar**

Step 5. Type following Program :

```
package PackageDemo;
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.GenericOptionsParser;
public class WordCount {
public static void main(String [] args) throws Exception
{
Configuration c=new Configuration();
String[] files=new GenericOptionsParser(c,args).getRemainingArgs();
Path input=new Path(files[0]);
Path output=new Path(files[1]);
Job j=new Job(c,"wordcount");
j.setJarByClass(WordCount.class);
j.setMapperClass(MapForWordCount.class);
```

```
j.setReducerClass(ReduceForWordCount.class);
j.setOutputKeyClass(Text.class);
j.setOutputValueClass(IntWritable.class);
FileInputFormat.addInputPath(j, input);
FileOutputFormat.setOutputPath(j, output);
System.exit(j.waitForCompletion(true)?0:1);
}
public static class MapForWordCount extends Mapper<LongWritable, Text, Text, Int
Writable>{
public void map(LongWritable key, Text value, Context con) throws IOException, Inte
rruptedException
{
String line = value.toString();
String[] words=line.split(",");
for(String word: words )
{
     Text outputKey = new Text(word.toUpperCase().trim());
  IntWritable outputValue = new IntWritable(1);
  con.write(outputKey, outputValue);
}
}
}
public static class ReduceForWordCount extends Reducer<Text, IntWritable, Text, Int
Writable>
{
public void reduce(Text word, Iterable<IntWritable> values, Context con) throws IOE
xception, InterruptedException
{
int sum = 0;
   for(IntWritable value : values)
   {
   sum += value.get();
   }
   con.write(word, new IntWritable(sum));
}
}
}
```

Explanation

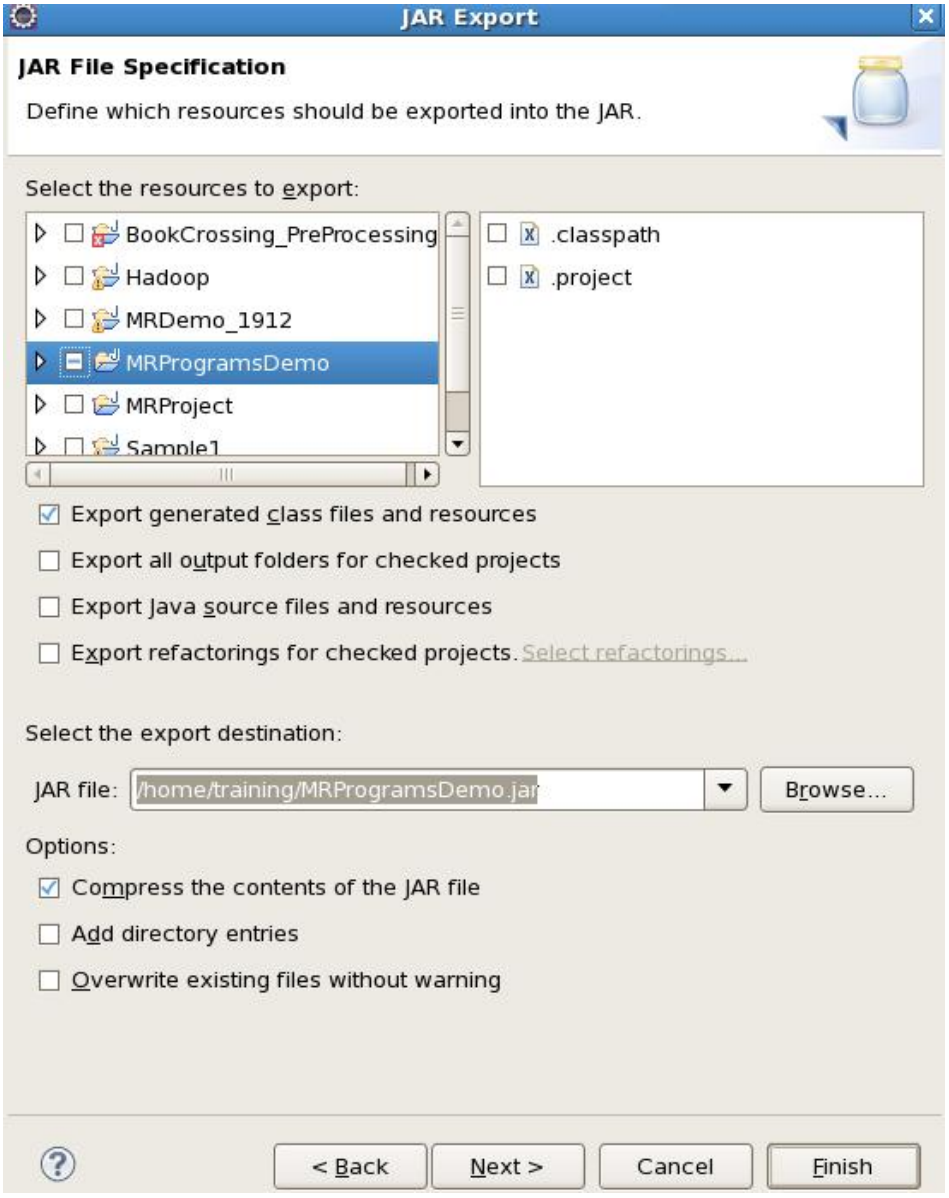
The program consist of 3 classes:

- Driver class (Public void static main- the entry point)

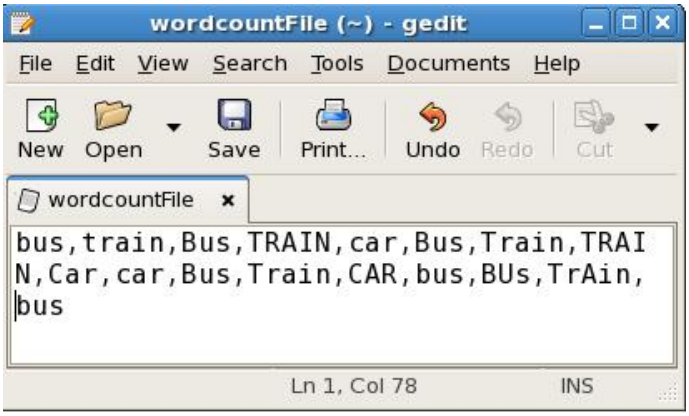
- Map class which **extends** public class Mapper<KEYIN,VALUEIN,KEYOUT,VALUEOUT> and implements the Map function.
- Reduce class which extends public class Reducer<KEYIN,VALUEIN,KEYOUT,VALUEOUT> and implements the Reduce function.

Step 6. Make Jar File

Right Click on Project> Export> Select export destination as **Jar File** > next> Finish



Step 7: Take a text file and move it in HDFS



To Move this into Hadoop directly, open the terminal and enter the following commands:

```
[training@localhost ~]$ hadoop fs -put wordcountFile wordCountFile
```

Step 8 . Run Jar file

(hadoop jar jarfilename.jar packageName.ClassName PathToInputTextFile PathToOutputDirectry)

```
[training@localhost ~]$ hadoop jar MRProgramsDemo.jar PackageDemo.WordCount
wordCountFile MRDir1
```

Step 9. Open Result

```
[training@localhost ~]$ hadoop fs -ls MRDir1
Found 3 items
-rw-r--r--   1 training supergroup          0 2016-02-23 03:36 /user/training/MRDir1/_SU
CCESS
drwxr-xr-x   - training supergroup          0 2016-02-23 03:36 /user/training/MRDir1/_lo
gs
-rw-r--r--   1 training supergroup         20 2016-02-23 03:36 /user/training/MRDir1/part
-r-00000
[training@localhost ~]$ hadoop fs -cat MRDir1/part-r-00000
BUS     7
CAR     4
TRAIN   6
```

Week 6: Write a Map Reduce program that mines weather data.Weather sensors collecting data every hour at many locations across the globe gather a large volume of log data, which is a good candidate for analysis with MapReduce, since it is semi structured and record-oriented.

```
import java.io.IOException;
import java.util.Iterator;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.conf.Configuration;
public class MyMaxMin {
    //Mapper
    /**
    *MaxTemperatureMapper class is static and extends Mapper abstract class
    having four hadoop generics type LongWritable, Text, Text, Text.
    */
    public static class MaxTemperatureMapper extends
            Mapper<LongWritable, Text, Text, Text> {
        /**
        * @method map
        * This method takes the input as text data type.
```

```
        * Now leaving the first five tokens,it takes 6th token is taken as temp_max
and

        * 7th token is taken as temp_min. Now temp_max > 35 and temp_min <
10 are passed to the reducer.

        */

        @Override

        public void map(LongWritable arg0, Text Value, Context context)

                throws IOException, InterruptedException {

        //Converting the record (single line) to String and storing it in a String
variable line

            String line = Value.toString();

        //Checking if the line is not empty

            if (!(line.length() == 0)) {

                //date

                String date = line.substring(6, 14);

                //maximum temperature

                float temp_Max = Float

                        .parseFloat(line.substring(39, 45).trim());

                //minimum temperature

                float temp_Min = Float

                        .parseFloat(line.substring(47, 53).trim());

            //if maximum temperature is greater than 35 , its a hot day

                if (temp_Max > 35.0) {

                    // Hot day

                    context.write(new Text("Hot Day " + date),

                            new Text(String.valueOf(temp_Max)));
```

```
                        }
                        //if minimum temperature is less than 10 , its a cold day
                        if (temp_Min < 10) {
                                // Cold day
                                context.write(new Text("Cold Day " + date),
                                                new Text(String.valueOf(temp_Min)));
                        }
                }
        }
}

//Reducer
        /**
        *MaxTemperatureReducer class is static and extends Reducer abstract class
        having four hadoop generics type Text, Text, Text, Text.
        */
        public static class MaxTemperatureReducer extends
                        Reducer<Text, Text, Text, Text> {
                /**
                * @method reduce
                * This method takes the input as key and list of values pair from mapper,
it does aggregation
                * based on keys and produces the final context.
                */
                public void reduce(Text Key, Iterator<Text> Values, Context context)
```

```
                    throws IOException, InterruptedException {
                //putting all the values in temperature variable of type String
                String temperature = Values.next().toString();
                context.write(Key, new Text(temperature));
            }
        }

        /**
        * @method main
        * This method is used for setting all the configuration properties.
        * It acts as a driver for map reduce code.
        */
        public static void main(String[] args) throws Exception {
                //reads the default configuration of cluster from the configuration xml
files
            Configuration conf = new Configuration();
            //Initializing the job with the default configuration of the cluster
            Job job = new Job(conf, "weather example");
            //Assigning the driver class name
            job.setJarByClass(MyMaxMin.class);
            //Key type coming out of mapper
            job.setMapOutputKeyClass(Text.class);
            //value type coming out of mapper
            job.setMapOutputValueClass(Text.class);
            //Defining the mapper class name
```

```
        job.setMapperClass(MaxTemperatureMapper.class);

        //Defining the reducer class name

        job.setReducerClass(MaxTemperatureReducer.class);

        //Defining input Format class which is responsible to parse the dataset into
a key value pair

        job.setInputFormatClass(TextInputFormat.class);

        //Defining output Format class which is responsible to parse the dataset
into a key value pair

        job.setOutputFormatClass(TextOutputFormat.class);

        //setting the second argument as a path in a path variable

        Path OutputPath = new Path(args[1]);

        //Configuring the input path from the filesystem into the job

        FileInputFormat.addInputPath(job, new Path(args[0]));

        //Configuring the output path from the filesystem into the job

        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        //deleting the context path automatically from hdfs so that we don't have
delete it explicitly

        OutputPath.getFileSystem(conf).delete(OutputPath);

        //exiting the job only if the flag value becomes false

        System.exit(job.waitForCompletion(true) ? 0 : 1);


    }
}
```

Step 2

Import the project in eclipse IDE in the same way it was told in earlier guide and change the jar paths with the jar files present in the lib directory of this project.

Step 3

When the project is not having any error, we will export it as a jar file, same as we did in wordcount mapreduce guide. Right Click on the Project file and click on Export. Select jar file.

Give the path where you want to save the file.Select the mail file by clicking on browse. Click on Finish to export.

Step 4

You can download the jar file directly using below link

temperature.jar

https://drive.google.com/file/d/0B2SFMPvhXPQ5RUlZZDZSR3FYVDA/view?usp=sharing

Download Dataset used by me using below link

weather_data.txt

https://drive.google.com/file/d/0B2SFMPvhXPQ5aFVILXAxbFh6ejA/view?usp=sharing

Step 5

Before running the mapreduce program to check what it does, see that your cluster is up and all the hadoop daemons are running.

Command: jps

Step 6

Send the weather dataset on to HDFS.

Command: hdfs dfs -put Downloads/weather_data.txt /

Command: hdfs dfs -ls /

Step 7

Run the jar file.

Command: hadoop jar temperature.jar /weather_data.txt /output_hotandcold

Step 8
Check output_hotandcold directory in HDFS.

Now inside part-r-00000 you will get your output.

Week 7: Implement Matrix Multiplication with Hadoop Map Reduce

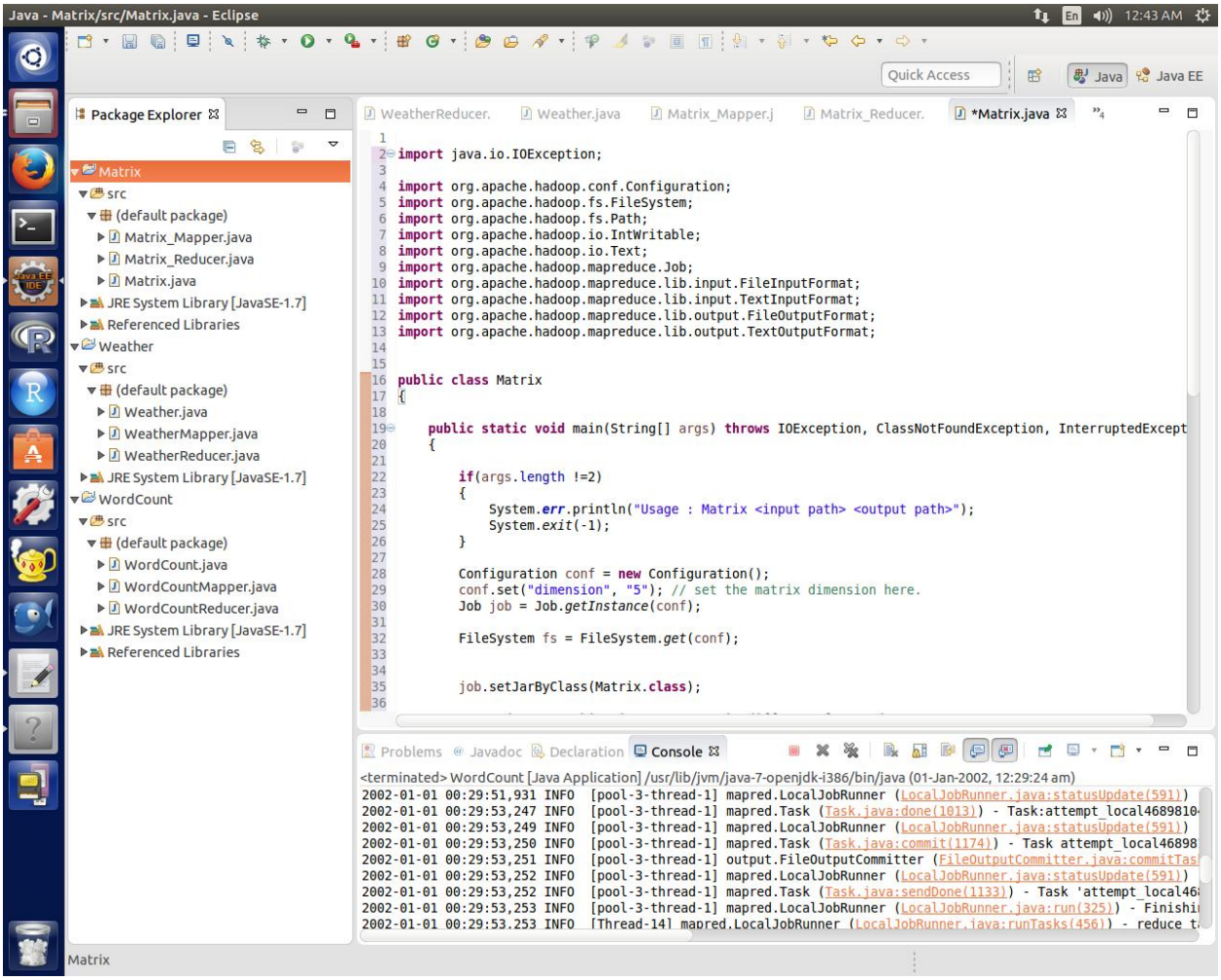
```
import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;


public class Matrix
{

        public static void main(String[] args) throws IOException,
ClassNotFoundException, InterruptedException
        {

                if(args.length !=2)
                {
                        System.err.println("Usage : Matrix <input path><output path>");
                        System.exit(-1);
                }

                Configuration conf = new Configuration();
                conf.set("dimension", "5"); // set the matrix dimension here.
                Job job = Job.getInstance(conf);
```

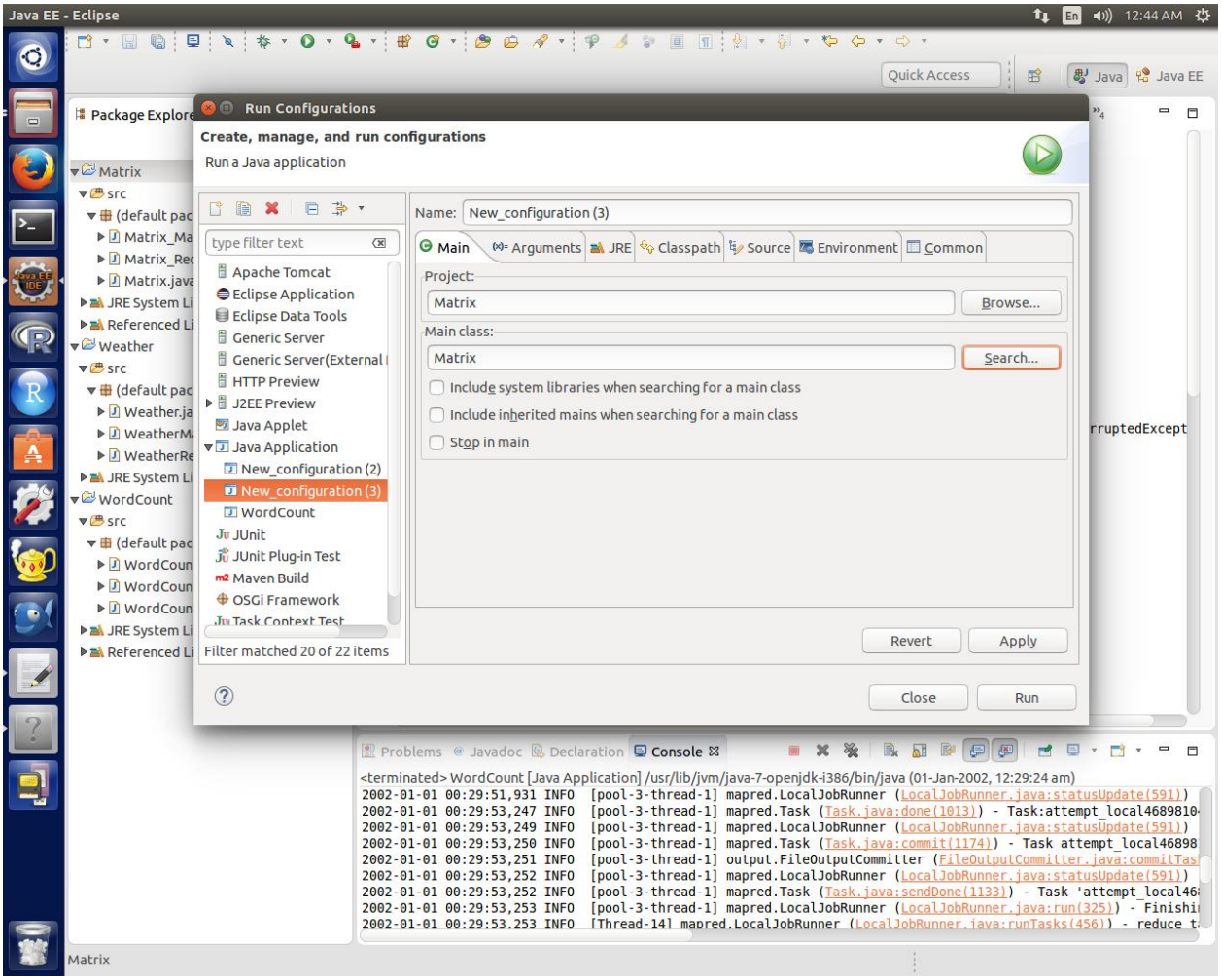
```
FileSystem fs = FileSystem.get(conf);


job.setJarByClass(Matrix.class);

// Need to set this since, map out is different from reduce out
job.setMapOutputKeyClass(Text.class);
job.setMapOutputValueClass(Text.class);

job.setOutputKeyClass(Text.class);
job.setOutputValueClass(IntWritable.class);

job.setMapperClass(Matrix_Mapper.class);
job.setReducerClass(Matrix_Reducer.class);

job.setInputFormatClass(TextInputFormat.class);
job.setOutputFormatClass(TextOutputFormat.class);

Path input = new Path(args[0]);
Path output = new Path(args[1]);

// Set the dimension of matrix


if(!fs.exists(input))
{
        System.err.println("Input file doesn't exists");
        return;
}
if(fs.exists(output))
{
        fs.delete(output, true);
        System.err.println("Output file deleted");
}
fs.close();

FileInputFormat.addInputPath(job, input);

FileOutputFormat.setOutputPath(job, output);

job.waitForCompletion(true);

System.out.println("MR Job Completed !");
```

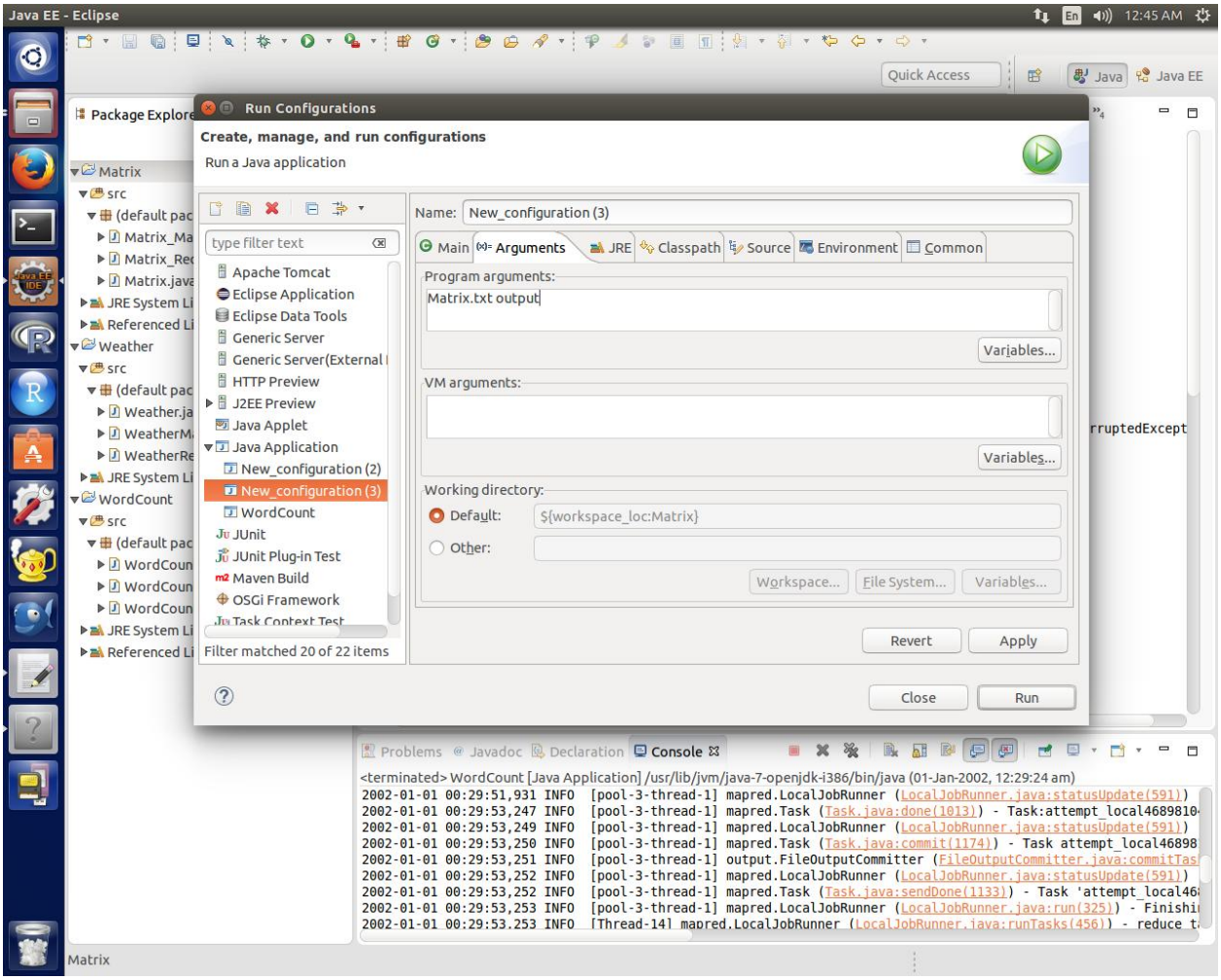
```
	}

}


import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class Matrix_Mapper extends Mapper<LongWritable,Text,Text,Text>
{

	/**
	 * Map function will collect/ group cell values required for
	 * calculating the output.
	 * @param key is ignored. Its just the byte offset
	 * @param value is a single line. (a, 0, 0, 63) (matrix name, row, column, value)
	 *
	 */

	@Override
	protected void map(LongWritable key, Text value,Context context)
					throws IOException, InterruptedException
	{
		System.out.println("Inside Map !");
		String line = value.toString();
		String[] entry = line.split(",");
		String sKey = "";
		String mat = entry[0].trim();

		String row, col;

		Configuration conf = context.getConfiguration();
		String dimension = conf.get("dimension");

		System.out.println("Dimension from Mapper = " + dimension);

		int dim = Integer.parseInt(dimension);
```

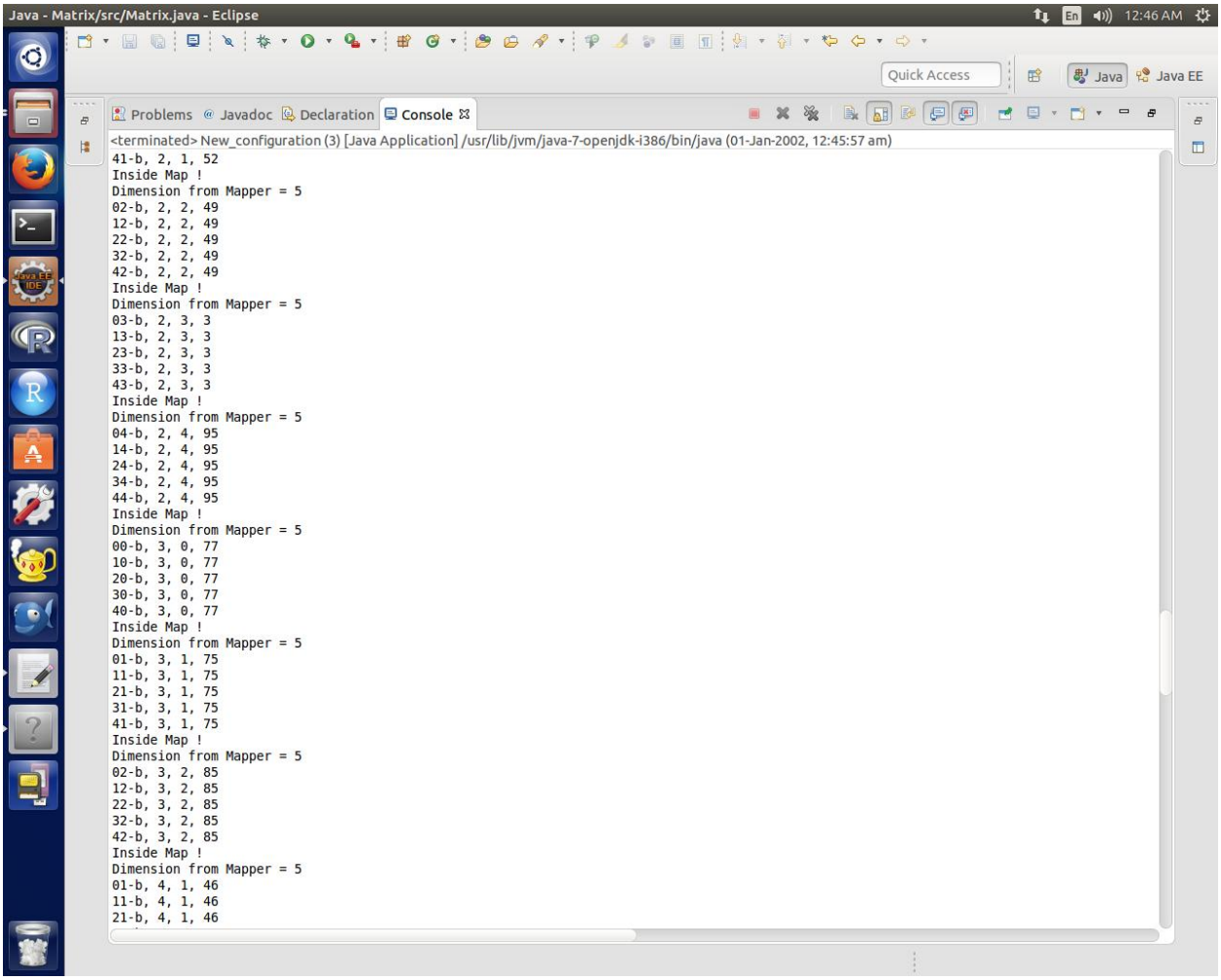
```
            if(mat.matches("a"))
            {
                for (int i =0; i < dim ; i++) // hard coding matrix size 5
                {
                    row = entry[1].trim(); // rowid
                    sKey = row+i;
                    System.out.println(sKey + "-" + value.toString());
                    context.write(new Text(sKey),value);
                }
            }

            if(mat.matches("b"))
            {
                for (int i =0; i < dim ; i++)
                {
                    col = entry[2].trim(); // colid
                    sKey = i+col;
                    System.out.println(sKey + "-" + value.toString());
                    context.write(new Text(sKey),value);
                }
            }

        }


}


import java.io.IOException;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class Matrix_Reducer extends Reducer<Text, Text, Text, IntWritable>
{

        /**
         * Reducer do the actual matrix multiplication.
         * @param key is the cell unique cell dimension (00) represents cell 0,0
         * @value values required to calculate matrix multiplication result of that cell.
         */
```

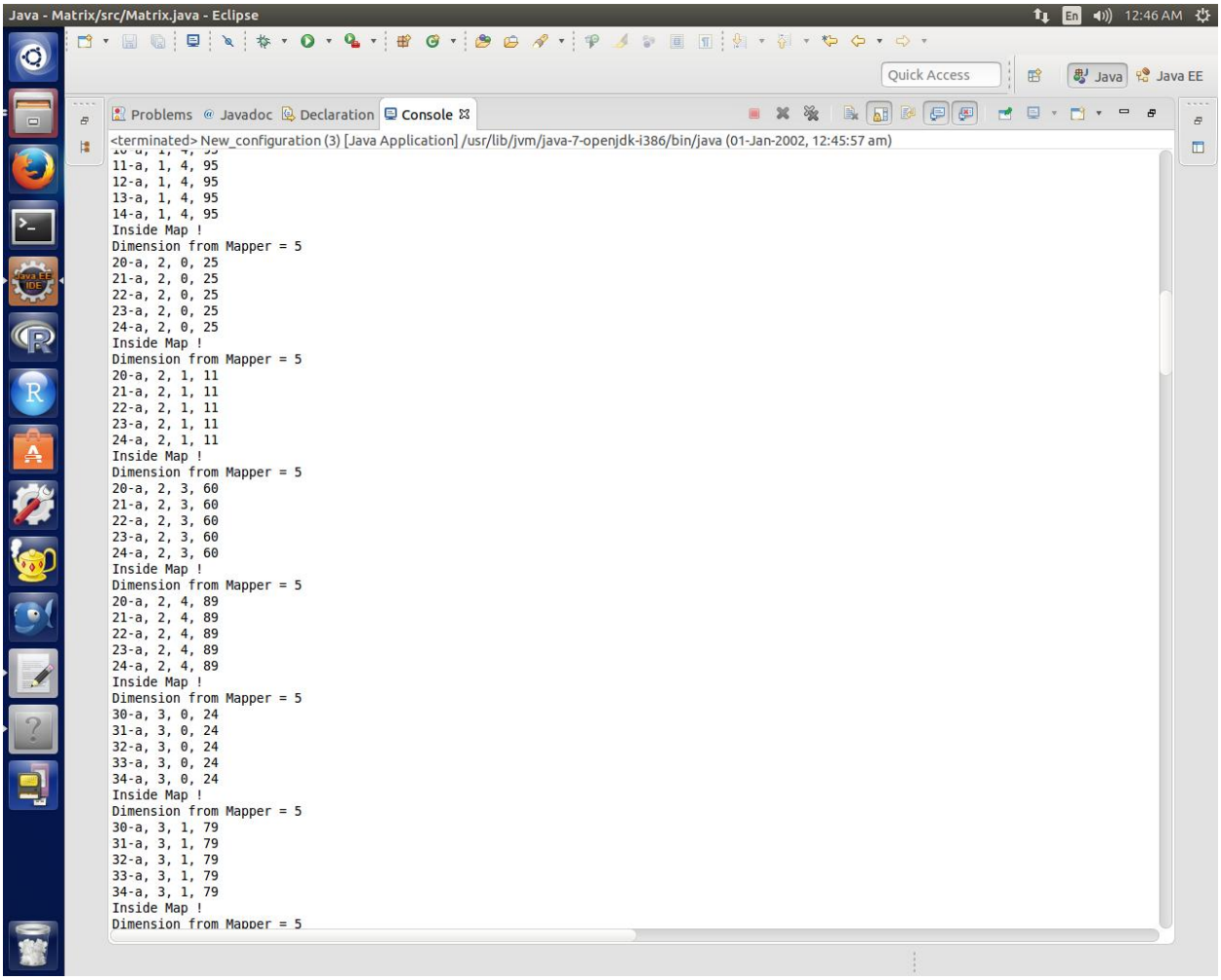
```
    @Override
    protected void reduce(Text key, Iterable<Text> values,Context context)
                                        throws IOException, InterruptedException
    {

        Configuration conf = context.getConfiguration();
        String dimension = conf.get("dimension");

        int dim = Integer.parseInt(dimension);

        //System.out.println("Dimension from Reducer = " + dimension);

        int[] row = new int[dim]; // hard coding as 5 X 5 matrix
        int[] col = new int[dim];

        for(Text val : values)
        {
            String[] entries = val.toString().split(",");
            if(entries[0].matches("a"))
            {
                int index = Integer.parseInt(entries[2].trim());
                row[index] = Integer.parseInt(entries[3].trim());
            }
            if(entries[0].matches("b"))
            {
                int index = Integer.parseInt(entries[1].trim());
                col[index] = Integer.parseInt(entries[3].trim());
            }
        }

        // Let us do matrix multiplication now..
        int total = 0;
        for(int i = 0 ; i < 5; i++)
        {
            total += row[i]*col[i];
        }
        System.out.println(key.toString() + "-" + total );
        context.write(key, new IntWritable(total));

    }

}
```

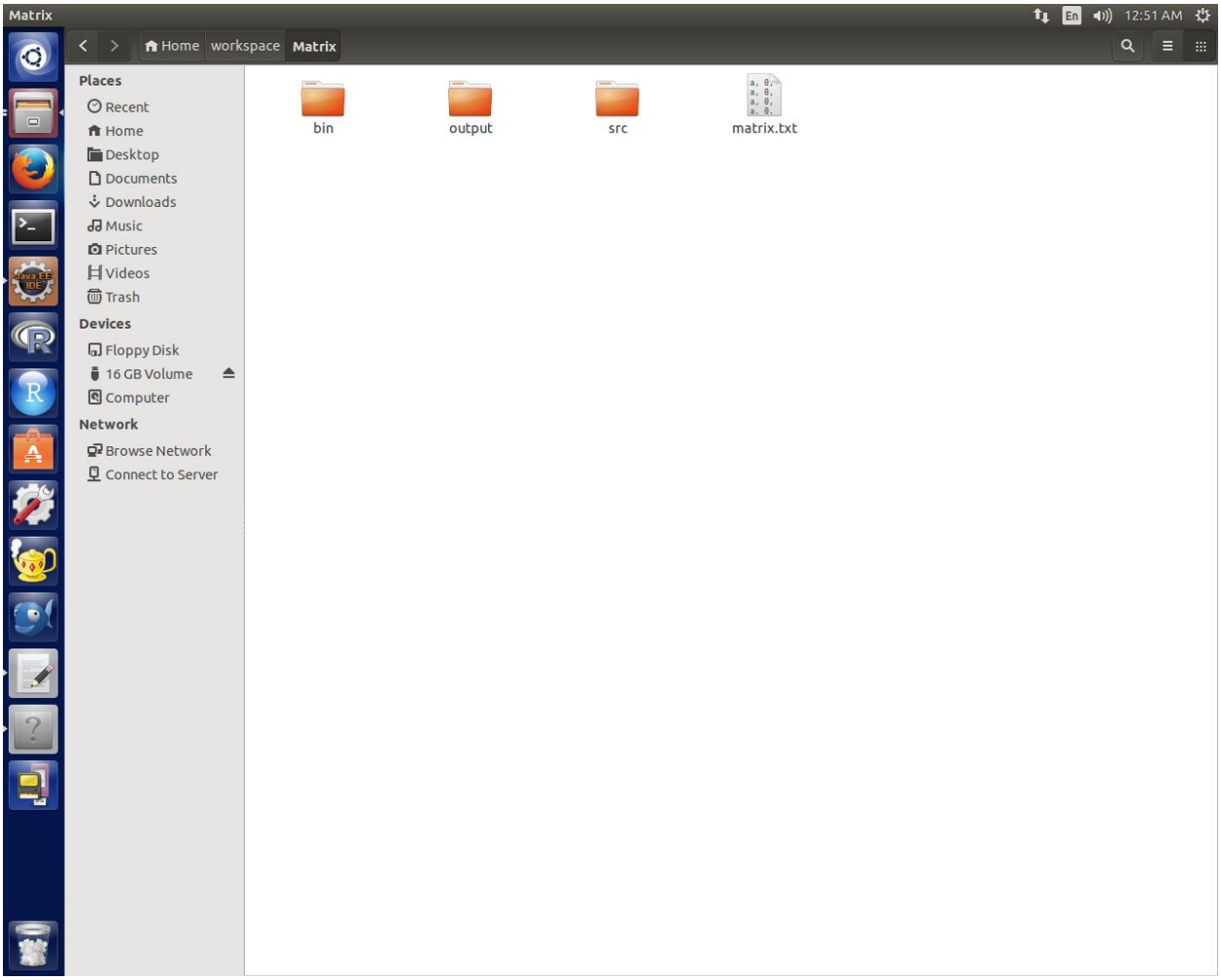

Java - Matrix/src/Matrix.java - Eclipse
Package Explorer
Matrix
src
(default package)
Matrix_Mapper.java
Matrix_Reducer.java
Matrix.java
JRE System Library [JavaSE-1.7]
Referenced Libraries
Weather
src
(default package)
Weather.java
WeatherMapper.java
WeatherReducer.java
JRE System Library [JavaSE-1.7]
WordCount
src
(default package)
WordCount.java
WordCountMapper.java
WordCountReducer.java
JRE System Library [JavaSE-1.7]
Referenced Libraries
WeatherReducer.
Weather.java
Matrix_Mapper.j
Matrix_Reducer.
*Matrix.java
import java.io.IOException;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;
public class Matrix
{
public static void main(String[] args) throws IOException, ClassNotFoundException, InterruptedExcept
{
if(args.length !=2)
{
System.err.println("Usage : Matrix <input path> <output path>");
System.exit(-1);
}
Configuration conf = new Configuration();
conf.set("dimension", "5"); // set the matrix dimension here.
Job job = Job.getInstance(conf);
FileSystem fs = FileSystem.get(conf);
job.setJarByClass(Matrix.class);
Problems
Javadoc
Declaration
Console
<terminated> WordCount [Java Application] /usr/lib/jvm/java-7-openjdk-i386/bin/java (01-Jan-2002, 12:29:24 am)
2002-01-01 00:29:51,931 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,247 INFO [pool-3-thread-1] mapred.Task (Task.java:done(1013)) - Task:attempt_local4689810
2002-01-01 00:29:53,249 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,250 INFO [pool-3-thread-1] mapred.Task (Task.java:commit(1174)) - Task attempt_local46898
2002-01-01 00:29:53,251 INFO [pool-3-thread-1] output.FileOutputCommitter (FileOutputCommitter.java:commitTas
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.Task (Task.java:sendDone(1133)) - Task 'attempt_local46
2002-01-01 00:29:53,253 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:run(325)) - Finishi
2002-01-01 00:29:53,253 INFO [Thread-14] mapred.LocalJobRunner (LocalJobRunner.java:runTasks(456)) - reduce t
Matrix

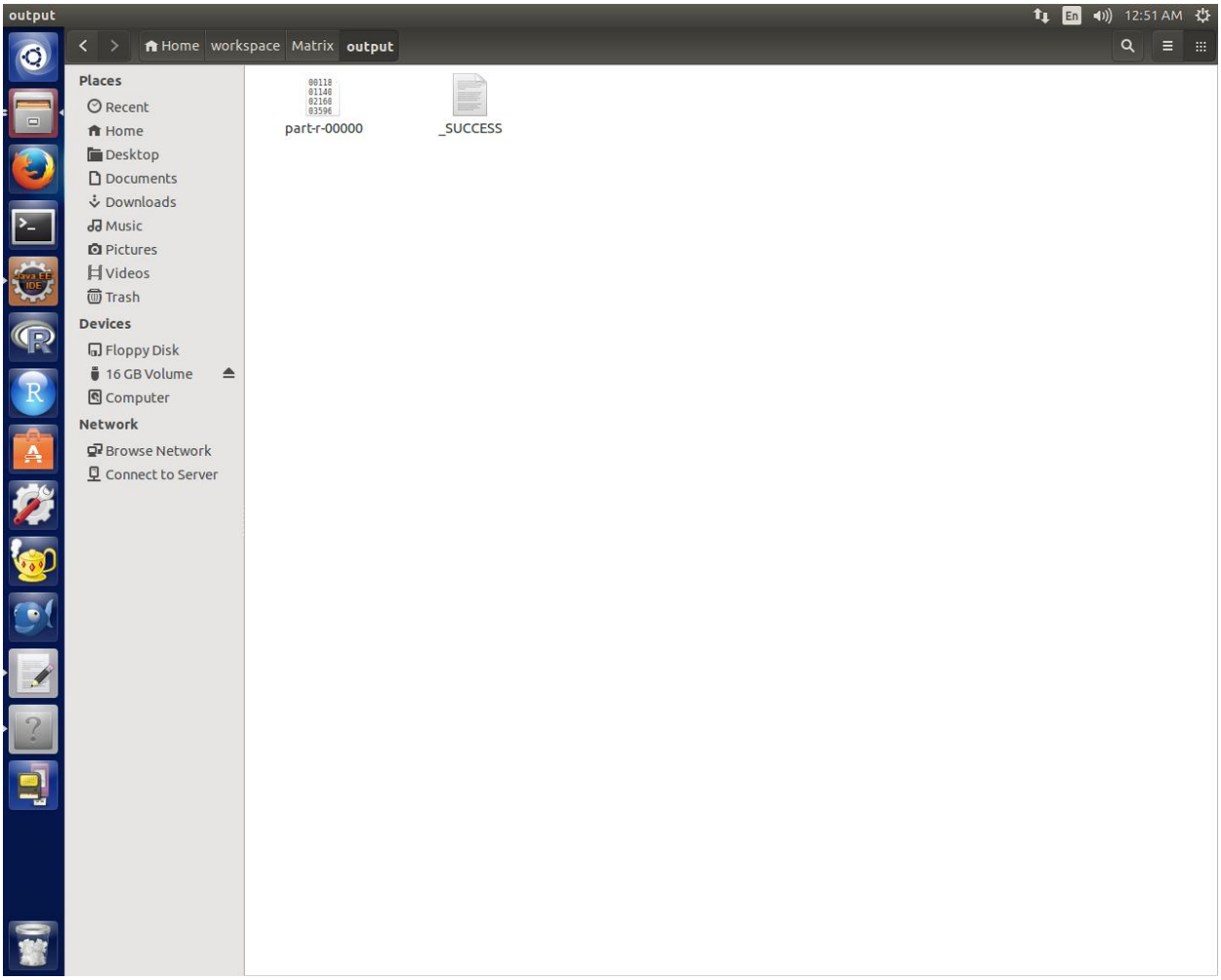

Java EE - Eclipse
Run Configurations
Create, manage, and run configurations
Run a Java application
Name: New_configuration (3)
Main
Arguments
JRE
Classpath
Source
Environment
Common
Project:
Matrix
Browse...
Main class:
Matrix
Search...
Include system libraries when searching for a main class
Include inherited mains when searching for a main class
Stop in main
Apache Tomcat
Eclipse Application
Eclipse Data Tools
Generic Server
HTTP Preview
J2EE Preview
Java Applet
Java Application
New_configuration (2)
New_configuration (3)
WordCount
JUnit
JUnit Plug-in Test
Maven Build
OSGi Framework
Filter matched 20 of 22 items
Revert
Apply
Close
Run
Problems
Javadoc
Declaration
Console
<terminated> WordCount [Java Application] /usr/lib/jvm/java-7-openjdk-i386/bin/java (01-Jan-2002, 12:29:24 am)
2002-01-01 00:29:51,931 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,247 INFO [pool-3-thread-1] mapred.Task (Task.java:done(1013)) - Task:attempt_local4689810
2002-01-01 00:29:53,249 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,250 INFO [pool-3-thread-1] mapred.Task (Task.java:commit(1174)) - Task attempt_local46898
2002-01-01 00:29:53,251 INFO [pool-3-thread-1] output.FileOutputCommitter (FileOutputCommitter.java:commitTas
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.Task (Task.java:sendDone(1133)) - Task 'attempt_local46
2002-01-01 00:29:53,253 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:run(325)) - Finishi
2002-01-01 00:29:53,253 INFO [Thread-14] mapred.LocalJobRunner (LocalJobRunner.java:runTasks(456)) - reduce t
Matrix

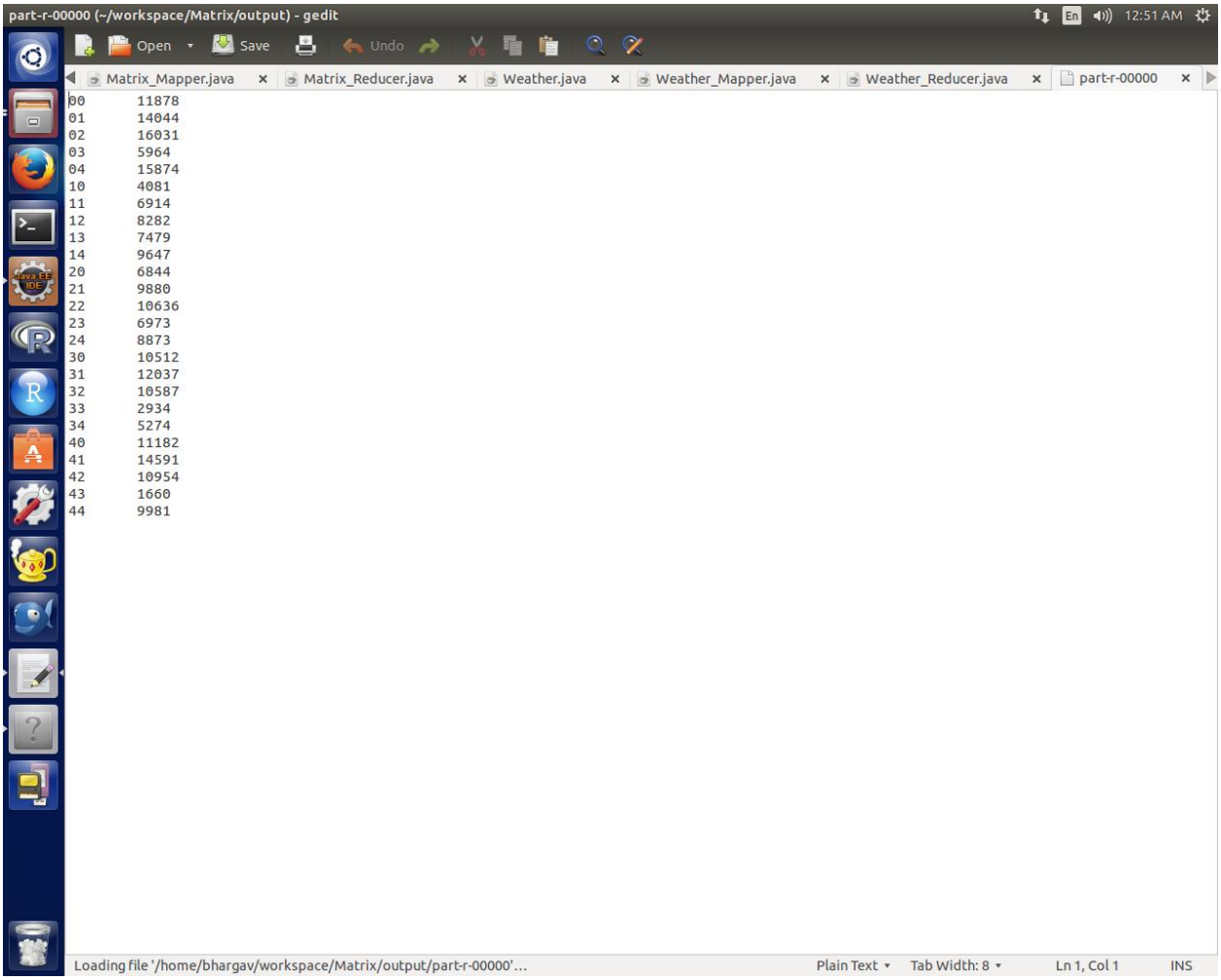

Java EE - Eclipse
Run Configurations
Create, manage, and run configurations
Run a Java application
Name: New_configuration (3)
Main
Arguments
JRE
Classpath
Source
Environment
Common
Program arguments:
Matrix.txt output
Variables...
VM arguments:
Working directory:
Default:
${workspace_loc:Matrix}
Other:
Workspace...
File System...
Variables...
Revert
Apply
Close
Run
Apache Tomcat
Eclipse Application
Eclipse Data Tools
Generic Server
HTTP Preview
J2EE Preview
Java Applet
Java Application
New_configuration (2)
New_configuration (3)
WordCount
JUnit
JUnit Plug-in Test
Maven Build
OSGi Framework
Filter matched 20 of 22 items
Problems
Javadoc
Declaration
Console
<terminated> WordCount [Java Application] /usr/lib/jvm/java-7-openjdk-i386/bin/java (01-Jan-2002, 12:29:24 am)
2002-01-01 00:29:51,931 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,247 INFO [pool-3-thread-1] mapred.Task (Task.java:done(1013)) - Task:attempt_local4689810
2002-01-01 00:29:53,249 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,250 INFO [pool-3-thread-1] mapred.Task (Task.java:commit(1174)) - Task attempt_local46898
2002-01-01 00:29:53,251 INFO [pool-3-thread-1] output.FileOutputCommitter (FileOutputCommitter.java:commitTas
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:statusUpdate(591))
2002-01-01 00:29:53,252 INFO [pool-3-thread-1] mapred.Task (Task.java:sendDone(1133)) - Task 'attempt_local46
2002-01-01 00:29:53,253 INFO [pool-3-thread-1] mapred.LocalJobRunner (LocalJobRunner.java:run(325)) - Finishi
2002-01-01 00:29:53,253 INFO [Thread-14] mapred.LocalJobRunner (LocalJobRunner.java:runTasks(456)) - reduce t
Matrix


Java - Matrix/src/Matrix.java - Eclipse
12:46 AM
Quick Access
Java
Java EE
Problems
Javadoc
Declaration
Console
<terminated> New_configuration (3) [Java Application] /usr/lib/jvm/java-7-openjdk-i386/bin/java (01-Jan-2002, 12:45:57 am)
41-b, 2, 1, 52
Inside Map !
Dimension from Mapper = 5
02-b, 2, 2, 49
12-b, 2, 2, 49
22-b, 2, 2, 49
32-b, 2, 2, 49
42-b, 2, 2, 49
Inside Map !
Dimension from Mapper = 5
03-b, 2, 3, 3
13-b, 2, 3, 3
23-b, 2, 3, 3
33-b, 2, 3, 3
43-b, 2, 3, 3
Inside Map !
Dimension from Mapper = 5
04-b, 2, 4, 95
14-b, 2, 4, 95
24-b, 2, 4, 95
34-b, 2, 4, 95
44-b, 2, 4, 95
Inside Map !
Dimension from Mapper = 5
00-b, 3, 0, 77
10-b, 3, 0, 77
20-b, 3, 0, 77
30-b, 3, 0, 77
40-b, 3, 0, 77
Inside Map !
Dimension from Mapper = 5
01-b, 3, 1, 75
11-b, 3, 1, 75
21-b, 3, 1, 75
31-b, 3, 1, 75
41-b, 3, 1, 75
Inside Map !
Dimension from Mapper = 5
02-b, 3, 2, 85
12-b, 3, 2, 85
22-b, 3, 2, 85
32-b, 3, 2, 85
42-b, 3, 2, 85
Inside Map !
Dimension from Mapper = 5
01-b, 4, 1, 46
11-b, 4, 1, 46
21-b, 4, 1, 46

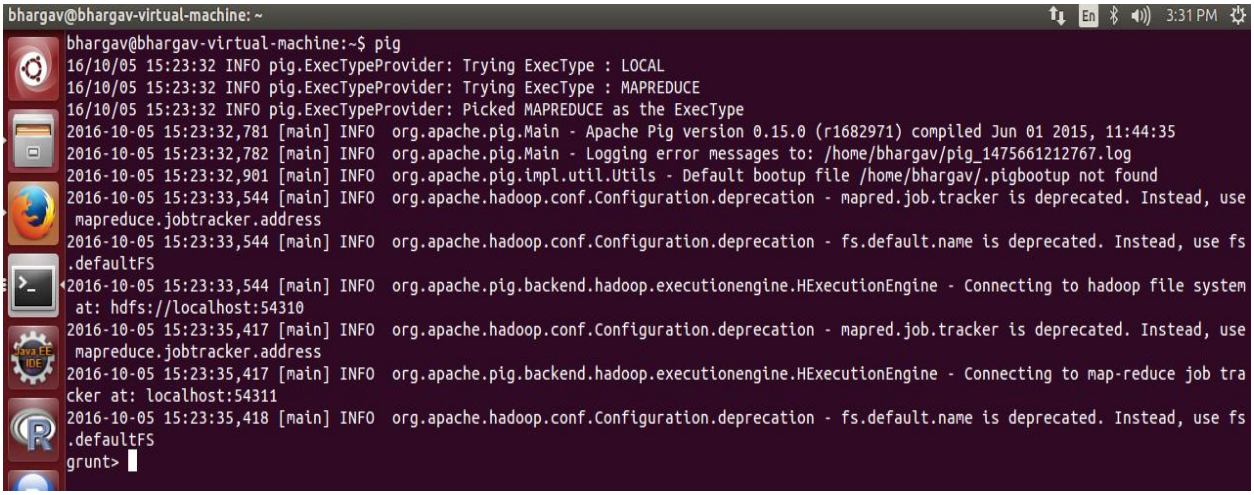

Java - Matrix/src/Matrix.java - Eclipse
12:46 AM
Quick Access
Java
Java EE
Problems
Javadoc
Declaration
Console
<terminated> New_configuration (3) [Java Application] /usr/lib/jvm/java-7-openjdk-i386/bin/java (01-Jan-2002, 12:45:57 am)
11-a, 1, 4, 95
12-a, 1, 4, 95
13-a, 1, 4, 95
14-a, 1, 4, 95
Inside Map !
Dimension from Mapper = 5
20-a, 2, 0, 25
21-a, 2, 0, 25
22-a, 2, 0, 25
23-a, 2, 0, 25
24-a, 2, 0, 25
Inside Map !
Dimension from Mapper = 5
20-a, 2, 1, 11
21-a, 2, 1, 11
22-a, 2, 1, 11
23-a, 2, 1, 11
24-a, 2, 1, 11
Inside Map !
Dimension from Mapper = 5
20-a, 2, 3, 60
21-a, 2, 3, 60
22-a, 2, 3, 60
23-a, 2, 3, 60
24-a, 2, 3, 60
Inside Map !
Dimension from Mapper = 5
20-a, 2, 4, 89
21-a, 2, 4, 89
22-a, 2, 4, 89
23-a, 2, 4, 89
24-a, 2, 4, 89
Inside Map !
Dimension from Mapper = 5
30-a, 3, 0, 24
31-a, 3, 0, 24
32-a, 3, 0, 24
33-a, 3, 0, 24
34-a, 3, 0, 24
Inside Map !
Dimension from Mapper = 5
30-a, 3, 1, 79
31-a, 3, 1, 79
32-a, 3, 1, 79
33-a, 3, 1, 79
34-a, 3, 1, 79
Inside Map !
Dimension from Mapper = 5


Matrix
12:51 AM
Home
workspace
Matrix
Places
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Devices
Floppy Disk
16 GB Volume
Computer
Network
Browse Network
Connect to Server
bin
output
src
matrix.txt


output
12:51 AM
Home
workspace
Matrix
output
Places
Recent
Home
Desktop
Documents
Downloads
Music
Pictures
Videos
Trash
Devices
Floppy Disk
16 GB Volume
Computer
Network
Browse Network
Connect to Server
part-r-00000
_SUCCESS


part-r-00000 (~/workspace/Matrix/output) - gedit
12:51 AM
Open
Save
Undo
Matrix_Mapper.java
Matrix_Reducer.java
Weather.java
Weather_Mapper.java
Weather_Reducer.java
part-r-00000
00 11878
01 14044
02 16031
03 5964
04 15874
10 4081
11 6914
12 8282
13 7479
14 9647
20 6844
21 9880
22 10636
23 6973
24 8873
30 10512
31 12037
32 10587
33 2934
34 5274
40 11182
41 14591
42 10954
43 1660
44 9981
Loading file '/home/bhargav/workspace/Matrix/output/part-r-00000'...
Plain Text
Tab Width: 8
Ln 1, Col 1
INS

Week 8,9:Install and Run Pig then write Pig Latin scripts to sort, group, join, project, and filter your data.



Place Your File in HDFS For Executing in PIG:

```
bhargav@bhargav-virtual-machine:~$ hadoop fs -put /home/bhargav/names1.txt
put: `names1.txt': File exists
bhargav@bhargav-virtual-machine:~$ hadoop fs -put /home/bhargav/names2.txt /user
/bhargav/pig
put: `/user/bhargav/pig/names2.txt': File exists
bhargav@bhargav-virtual-machine:~$
```

LOAD:

```
grunt> d = load 'names2.txt' using PigStorage(' ') as (name: chararray, age: int);
2016-10-05 15:32:40,710 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.na
.defaultFS
2016-10-05 15:32:40,710 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.t
 mapreduce.jobtracker.address
```

$ gedit student

1,A

2,B

3,C

4,D

$hadoop fs -put student bda2

(= is preceded and succeeded by wide space)

grunt> s = load ‘bda2’ using PigStorage(‘,’) as (id:int name: chararray);

grunt> dump s;

FILTER:

```
grunt> f = filter d by age < 30;
grunt> dump f;
2016-10-05 15:33:05,726 [main] INFO  org.apache.pig.tools.pigstats.ScriptState - Pig features
2016-10-05 15:33:05,847 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.def
.defaultFS
2016-10-05 15:33:05,847 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred
 mapreduce.jobtracker.address
2016-10-05 15:33:05,864 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Key [pig.schemat
ode.
2016-10-05 15:33:05,955 [main] INFO  org.apache.pig.newplan.logical.optimizer.LogicalPlanOptim
nMapKeyPrune, ConstantCalculator, GroupByConstParallelSetter, LimitOptimizer, LoadTypeCastInse
onFilterOptimizer, PredicatePushdownOptimizer, PushDownForEachFlatten, PushUpFilter, SplitFilt
2016-10-05 15:33:06,440 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLa
shold: 100 optimistic? false
2016-10-05 15:33:06,518 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLa
efore optimization: 1
2016-10-05 15:33:06,519 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLa
fter optimization: 1
2016-10-05 15:33:06,611 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.def
.defaultFS
2016-10-05 15:33:06,713 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - sessio
rics.session.id
```

grunt> f = filter s by id>1;

grunt> dump f;

SORT:

Descending:

grunt> rsort = order s by id desc;

grunt>dump rsort;

Ascending:

grunt> sort = order s by id asc;

grunt> dump sort;

```
grunt> d = load 'names2.txt' using PigStorage(',') as (name: chararray, age: int);
2016-10-05 15:41:06,571 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs
.defaultFS
2016-10-05 15:41:06,572 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use
 mapreduce.jobtracker.address
grunt> sorted = order d by name ASC;
grunt> dump sorted;
```

```
Total bytes written : 42000347
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_local630342386_0002 ->      job_local826211907_0003,
job_local826211907_0003 ->      job_local633928928_0004,
job_local633928928_0004


2016-10-05 15:39:23,099 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,141 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,146 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,173 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,178 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,181 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,320 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,325 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,330 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JV
ker, sessionId= - already initialized
2016-10-05 15:39:23,359 [main] WARN  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.Ma
ing ACCESSING_NON_EXISTENT_FIELD 1 time(s).
2016-10-05 15:39:23,362 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.Ma
2016-10-05 15:39:23,363 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.n
.defaultFS
2016-10-05 15:39:23,367 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.t
 mapreduce.jobtracker.address
2016-10-05 15:39:23,375 [main] WARN  org.apache.pig.data.SchemaTupleBackend - SchemaTupleBackend has
2016-10-05 15:39:23,386 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total i
2016-10-05 15:39:23,386 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil -
(,)
(Krishna,30)
(Krishna,20)
(Lakshmi,18)
(Raju,15)
(Rama,25)
(Rama,25)
(Ravi,25)
(Sita,20)
```

DISTINCT:

grunt> d_d = DISTINCT s;

grunt> dump d_d;

For Each:

grunt> for_each = FOREACH s by id,name;

grunt> dump for_each

GROUP:

```
grunt> d = load 'names2.txt' using PigStorage(',') as (name: chararray, age: int);
2016-10-05 15:45:16,224 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs
.defaultFS
2016-10-05 15:45:16,224 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use
 mapreduce.jobtracker.address
grunt> grouped = group d by name;
grunt> dump grouped
```

grunt> g_d = GROUP s1 by age;

grunt> dump g_d;

```
2016-10-05 15:47:22,702 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2016-10-05 15:47:22,708 [main] INFO  org.apache.pig.tools.pigstats.mapreduce.SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
2.7.0   0.15.0  bhargav 2016-10-05 15:47:09     2016-10-05 15:47:22     GROUP_BY

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTime      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTimeM
edianReducetime Alias   Feature Outputs
job_local587487624_0006 1       1       n/a     n/a     n/a     n/a     n/a     n/a     n/a     n/a     d,grouped       GROUP_BY     h
dfs://localhost:54310/tmp/temp-1420043986/tmp-2044210627,

Input(s):
Successfully read 9 records (63001404 bytes) from: "hdfs://localhost:54310/user/bhargav/names2.txt"

Output(s):
Successfully stored 7 records (63000693 bytes) in: "hdfs://localhost:54310/tmp/temp-1420043986/tmp-2044210627"

Counters:
Total records written : 7
Total bytes written : 63000693
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_local587487624_0006


2016-10-05 15:47:22,713 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JVM Metrics with processName=JobTrac
ker, sessionId= - already initialized
2016-10-05 15:47:22,722 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JVM Metrics with processName=JobTrac
ker, sessionId= - already initialized
2016-10-05 15:47:22,724 [main] INFO  org.apache.hadoop.metrics.jvm.JvmMetrics - Cannot initialize JVM Metrics with processName=JobTrac
ker, sessionId= - already initialized
2016-10-05 15:47:22,748 [main] WARN  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Encountered Warn
ing ACCESSING_NON_EXISTENT_FIELD 1 time(s).
2016-10-05 15:47:22,748 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2016-10-05 15:47:22,749 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs
.defaultFS
2016-10-05 15:47:22,749 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use
 mapreduce.jobtracker.address
2016-10-05 15:47:22,750 [main] WARN  org.apache.pig.data.SchemaTupleBackend - SchemaTupleBackend has already been initialized
2016-10-05 15:47:22,814 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2016-10-05 15:47:22,814 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(Raju,{(Raju,15)})
(Rama,{(Rama,25),(Rama,25)})
(Ravi,{(Ravi,25)})
(Sita,{(Sita,20)})
(Krishna,{(Krishna,30),(Krishna,20)})
(Lakshmi,{(Lakshmi,18)})
(,{(,)})
```

JOIN:

$ gedit s1;

1,A

2,B

3,C

4,D

$ gedit s2;

1,A

2,B

3,C

4,D

grunt>j = join s1 by id, s2 by id;

grunt>dump j;

Right-outer join:

grunt>p1 = join s1 by id right outer, s2 by id;

grunt>dump p1;

Left-outer join:

grunt>p2 = JOIN s1 by id left outer, s2 by id;

grunt>dump p2;

```
grunt> d = load 'names1.txt' using PigStorage(',') as (name: chararray, age: int);
2016-10-05 15:50:39,291 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs
.defaultFS
2016-10-05 15:50:39,294 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use
 mapreduce.jobtracker.address
grunt> e = load 'names2.txt' using PigStorage(',') as (name: chararray, age: int);
2016-10-05 15:50:55,576 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs
.defaultFS
2016-10-05 15:50:55,576 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use
 mapreduce.jobtracker.address
grunt> f = JOIN d by age, e by age;
grunt> dump f;
```

```
2016-10-05 15:47:22,749 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tra
mapreduce.jobtracker.address
2016-10-05 15:47:22,750 [main] WARN  org.apache.pig.data.SchemaTupleBackend - SchemaTupleBackend has a
2016-10-05 15:47:22,814 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total inp
2016-10-05 15:47:22,814 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - T
(Raju,{(Raju,15)})
(Rama,{(Rama,25),(Rama,25)})
(Ravi,{(Ravi,25)})
(Sita,{(Sita,20)})
(Krishna,{(Krishna,30),(Krishna,20)})
(Lakshmi,{(Lakshmi,18)})
```

Aggregate Functions:

$gedit emp;

101,a,b,1000,40

102,c,d,1500,25

103,e,f,2000,24

104,g,h,1500,24

105,i,j,3000,25

$hadoop fs -put emp35 emp36

$pig

grunt>a = load ‘emp35’ using PigStorage(‘,’) as (f1:int, f2 : chararray, f3 : chararray, f4: int , f5:int);

grunt>dump a;

grunt>emp_grp = GROUP a all;

grunt>dump emp_grp;

SUM:

grunt> salary_sum = FOREACH emp_grp generate (a.f1,a.f2), SUM(a.f4);

grunt>dump salary_sum;

COUNT:
grunt>emp_count = foreach emp_grp generate (a.f1);

grunt>dump emp_count;

AVERAGE:

grunt> emp_avg = foreach emp_grp generate (a.f2,a.f4), AVG(a.f4);

grunt>gump emp_avg;

MAXIMUM:

grunt> emp_name = foreach emp_grp generate (a.f2,a.f4), MAX(a.f4);

grunt>dump emp_name;

MINIMUM:

grunt>emp_min = foreach emp_grp generate (a.f2,a.f4), MIN(a.f4);

grunt>dump emp_min;

UPPER:

grunt>emp_upr = foreach a generate (f1,f2), UPPER(f2);

grunt>dump emp_upr;

LOWER:

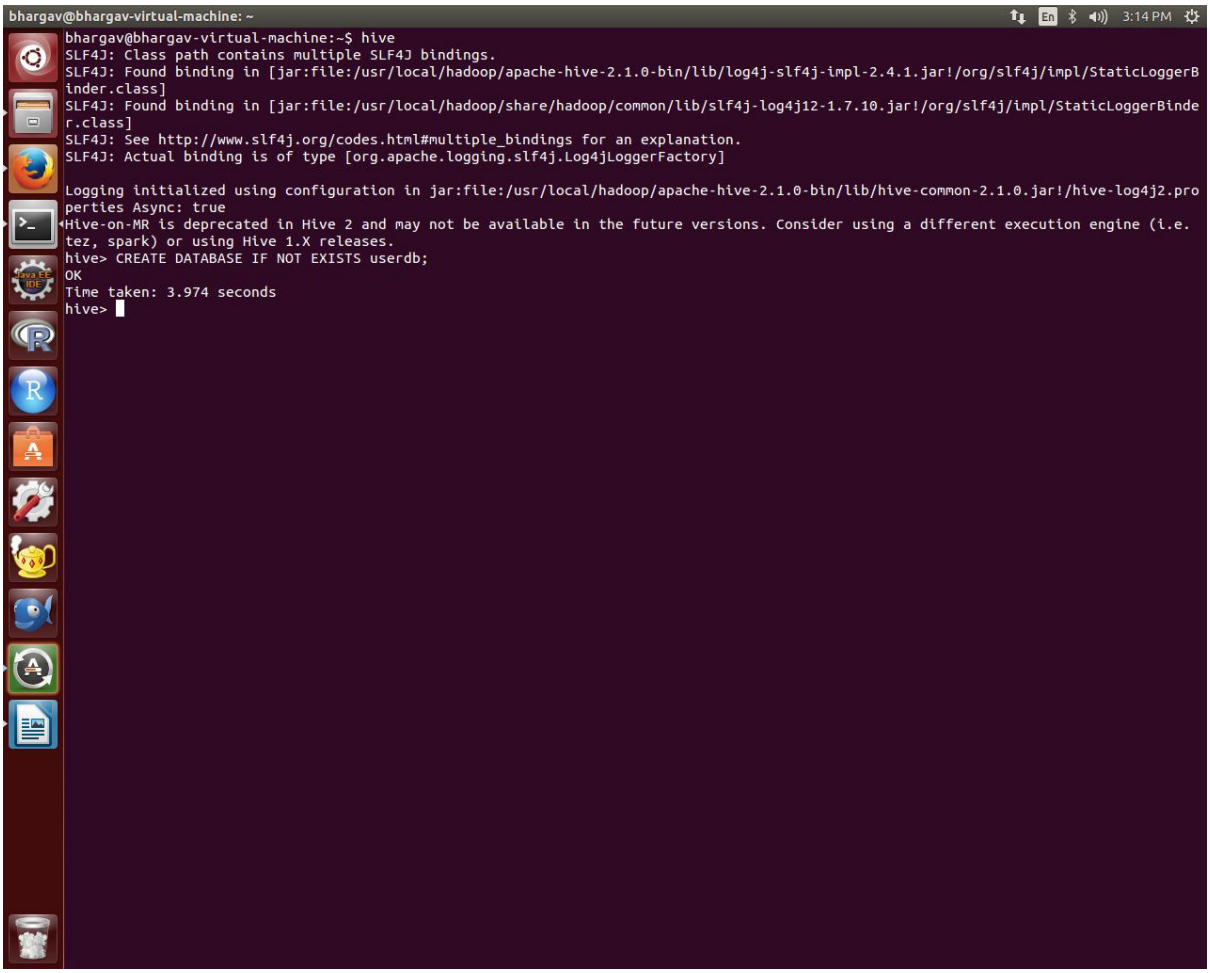
grunt>emp_lwr = foreach a generate (f1,f2), LOWER(f2);

grunt>dump emp_lwr;

Week 10,11: Install and Run HIVE then use Hive to Create, alter, and Drop Databases, Tables, Views, Functions and Indexes

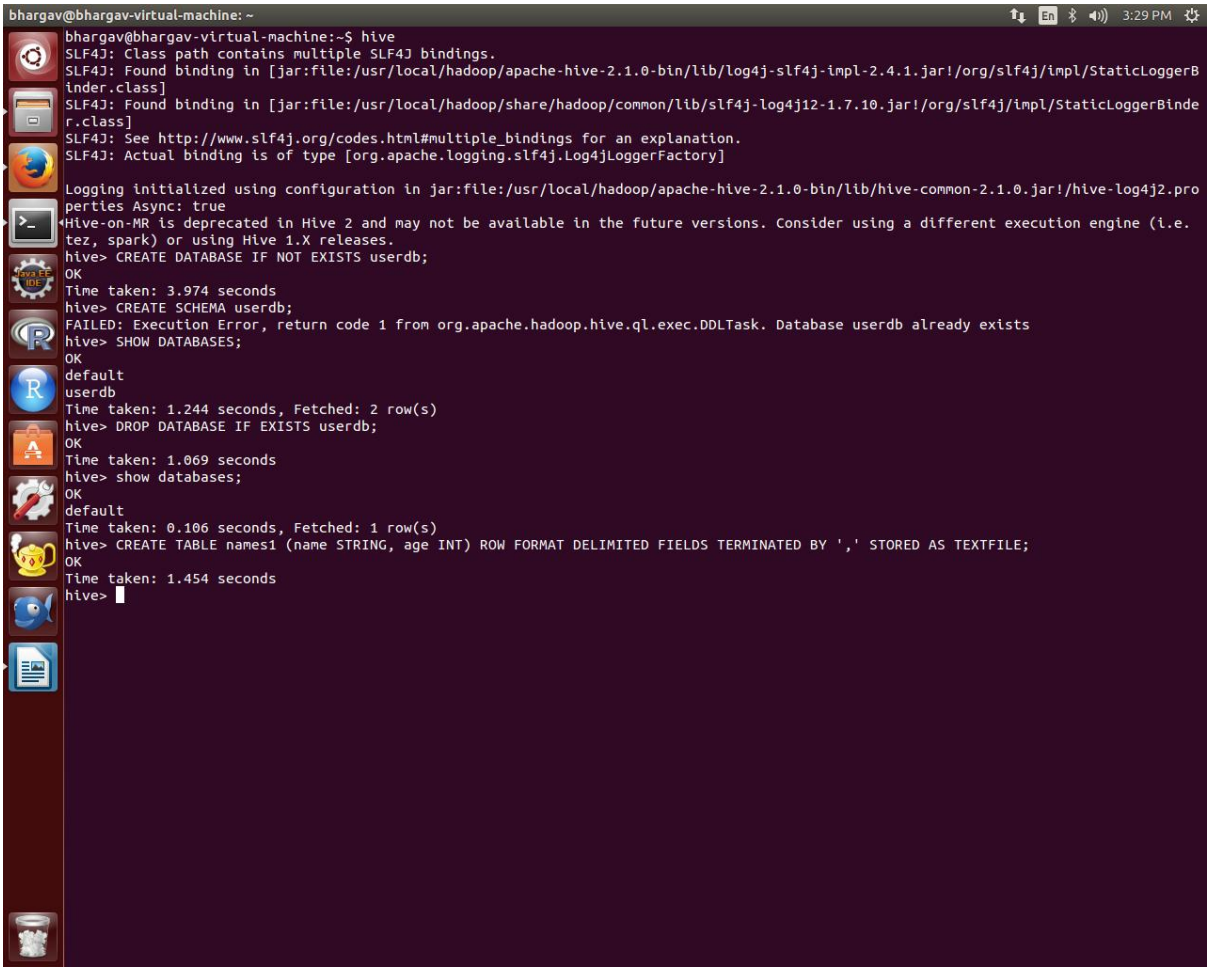
Create database

```
CREATE DATABASE IF NOT EXISTSuserdb;
CREATE SCHEMA userdb;

SHOW DATABASES;
```

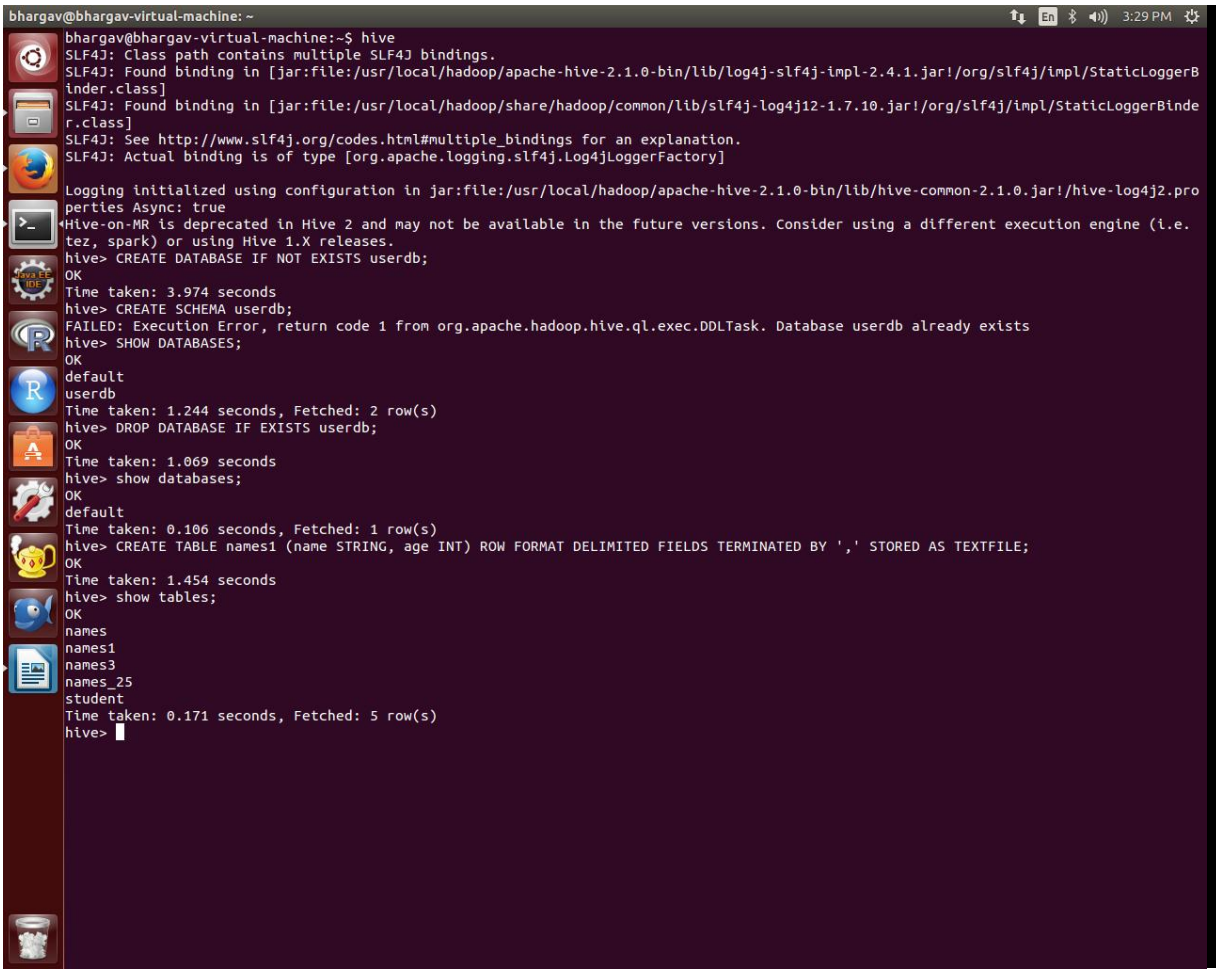


Create table

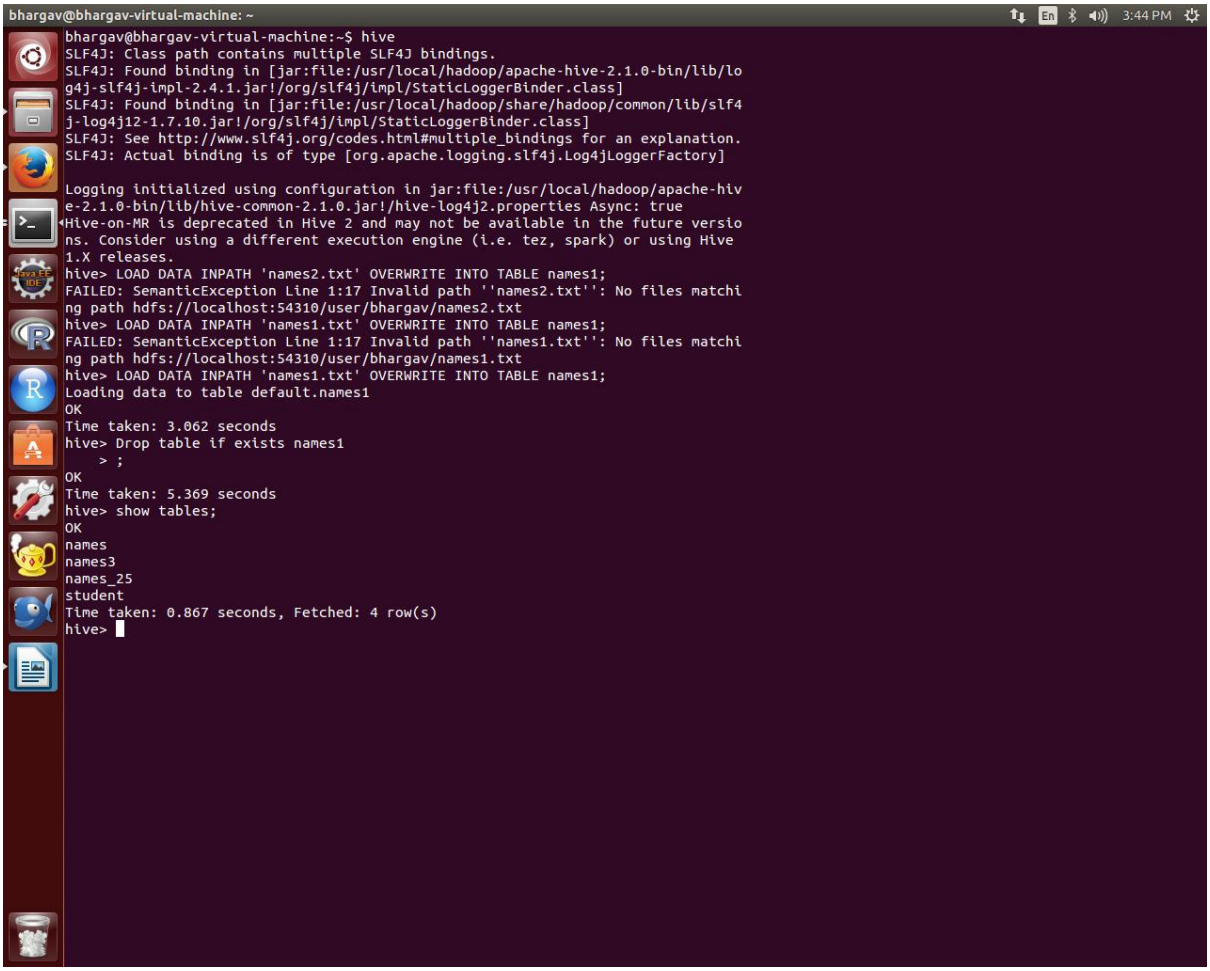
```
CREATE TABLE names1 (name STRING, age INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;
```

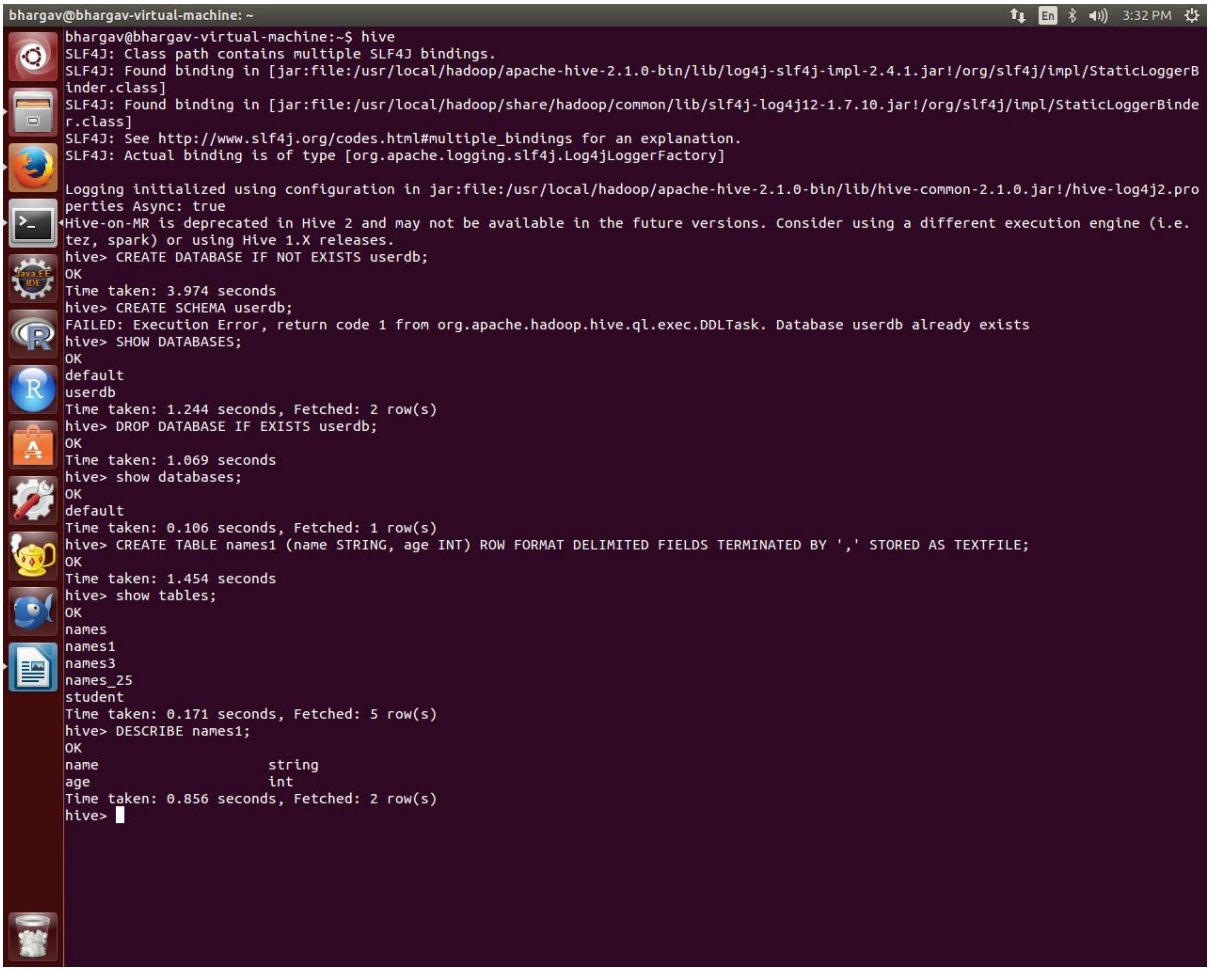

bhargav@bhargav-virtual-machine: ~
3:29 PM
bhargav@bhargav-virtual-machine:~$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/apache-hive-2.1.0-bin/lib/log4j-slf4j-impl-2.4.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Logging initialized using configuration in jar:file:/usr/local/hadoop/apache-hive-2.1.0-bin/lib/hive-common-2.1.0.jar!/hive-log4j2.properties Async: true
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez, spark) or using Hive 1.X releases.
hive> CREATE DATABASE IF NOT EXISTS userdb;
OK
Time taken: 3.974 seconds
hive> CREATE SCHEMA userdb;
FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.ql.exec.DDLTask. Database userdb already exists
hive> SHOW DATABASES;
OK
default
userdb
Time taken: 1.244 seconds, Fetched: 2 row(s)
hive> DROP DATABASE IF EXISTS userdb;
OK
Time taken: 1.069 seconds
hive> show databases;
OK
default
Time taken: 0.106 seconds, Fetched: 1 row(s)
hive> CREATE TABLE names1 (name STRING, age INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;
OK
Time taken: 1.454 seconds
hive>

SHOW
TABLES:

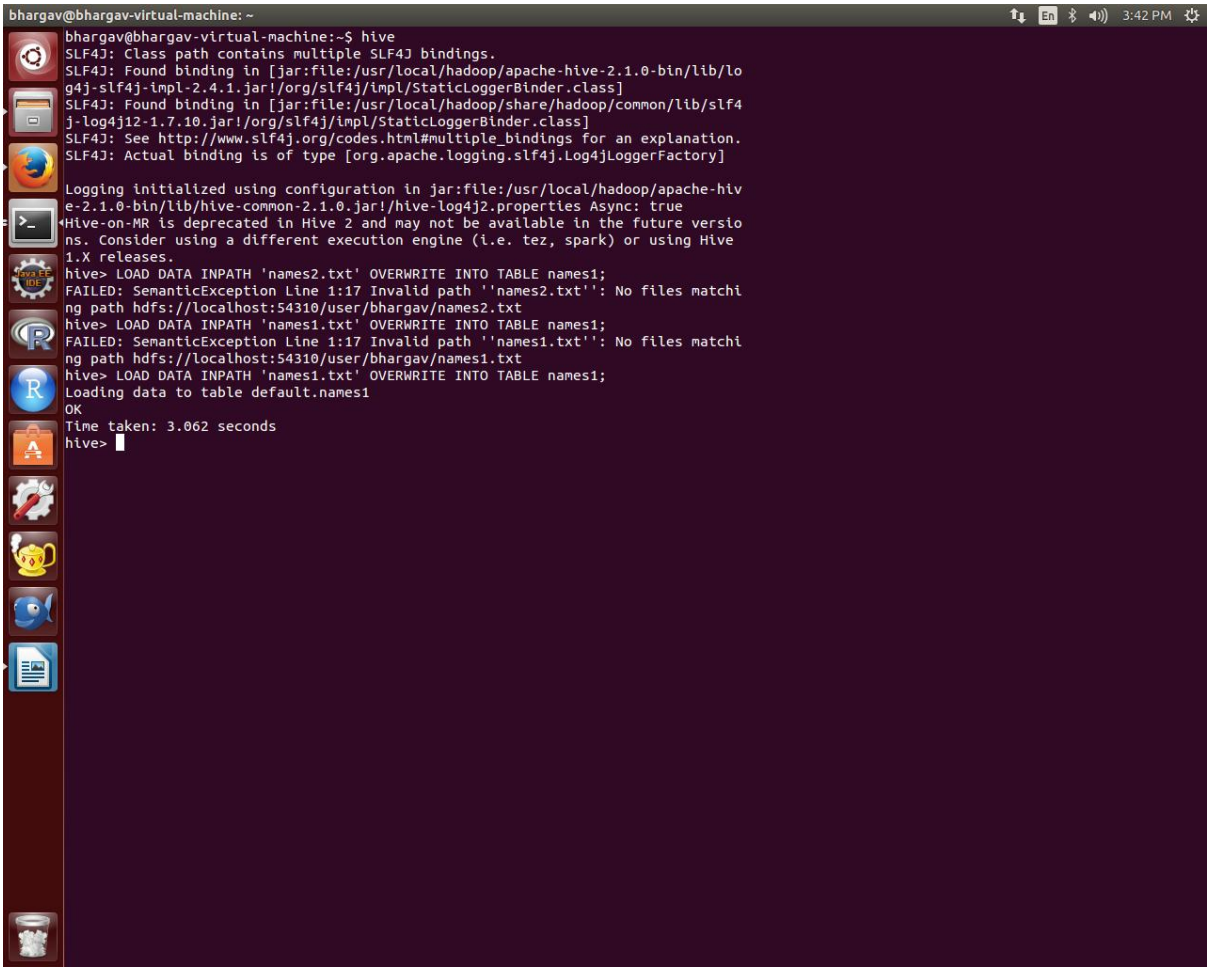

bhargav@bhargav-virtual-machine: ~
3:29 PM
bhargav@bhargav-virtual-machine:~$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/usr/local/hadoop/apache-hive-2.1.0-bin/lib/log4j-slf4j-impl-2.4.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/usr/local/hadoop/share/hadoop/common/lib/slf4j-log4j12-1.7.10.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Logging initialized using configuration in jar:file:/usr/local/hadoop/apache-hive-2.1.0-bin/lib/hive-common-2.1.0.jar!/hive-log4j2.properties Async: true
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. tez, spark) or using Hive 1.X releases.
hive> CREATE DATABASE IF NOT EXISTS userdb;
OK
Time taken: 3.974 seconds
hive> CREATE SCHEMA userdb;
FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.ql.exec.DDLTask. Database userdb already exists
hive> SHOW DATABASES;
OK
default
userdb
Time taken: 1.244 seconds, Fetched: 2 row(s)
hive> DROP DATABASE IF EXISTS userdb;
OK
Time taken: 1.069 seconds
hive> show databases;
OK
default
Time taken: 0.106 seconds, Fetched: 1 row(s)
hive> CREATE TABLE names1 (name STRING, age INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;
OK
Time taken: 1.454 seconds
hive> show tables;
OK
names
names1
names3
names_25
student
Time taken: 0.171 seconds, Fetched: 5 row(s)
hive>

DESCRIBE Names;



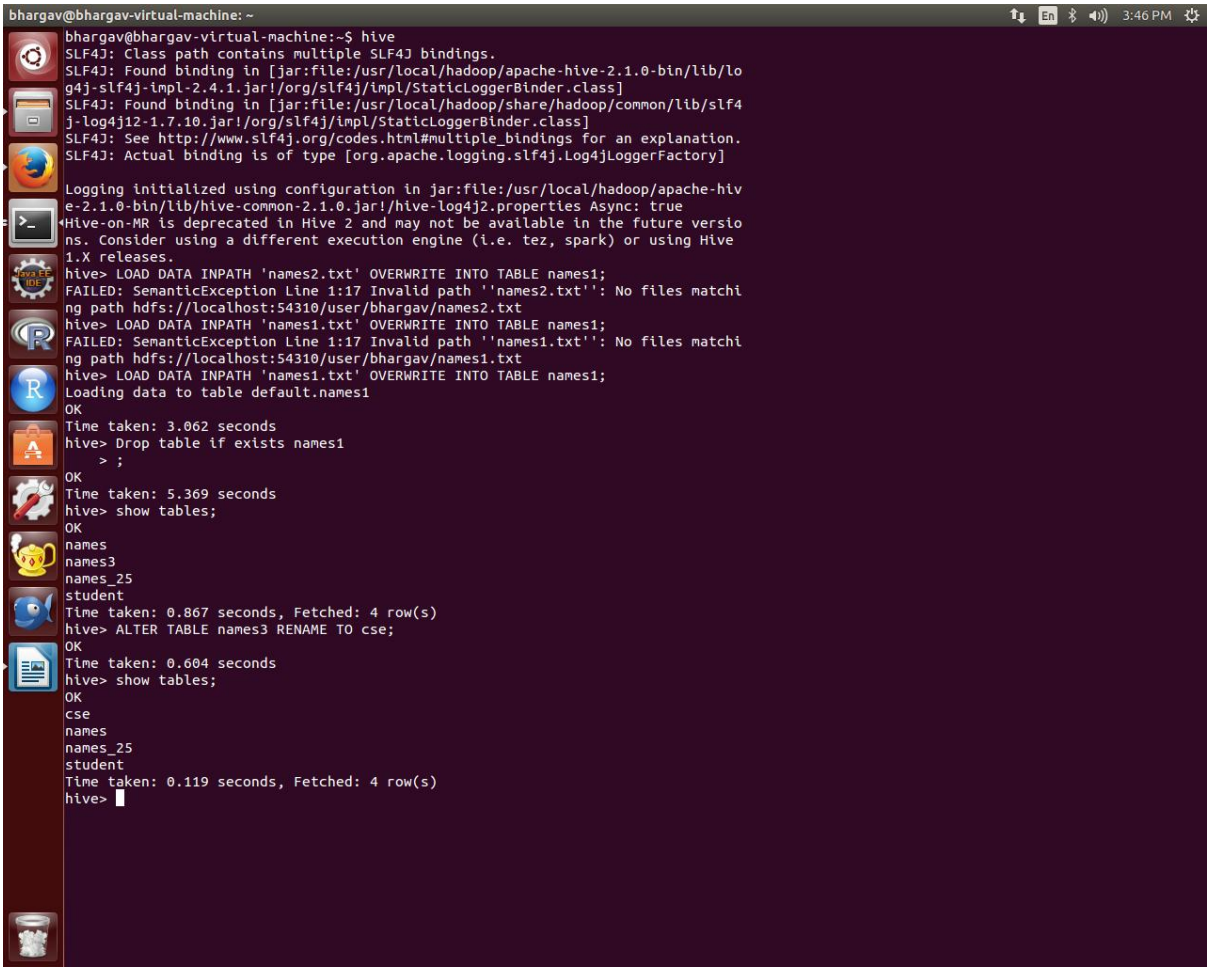


LOAD DATA INPATH 'names2.txt' OVERWRITE INTO TABLE names;



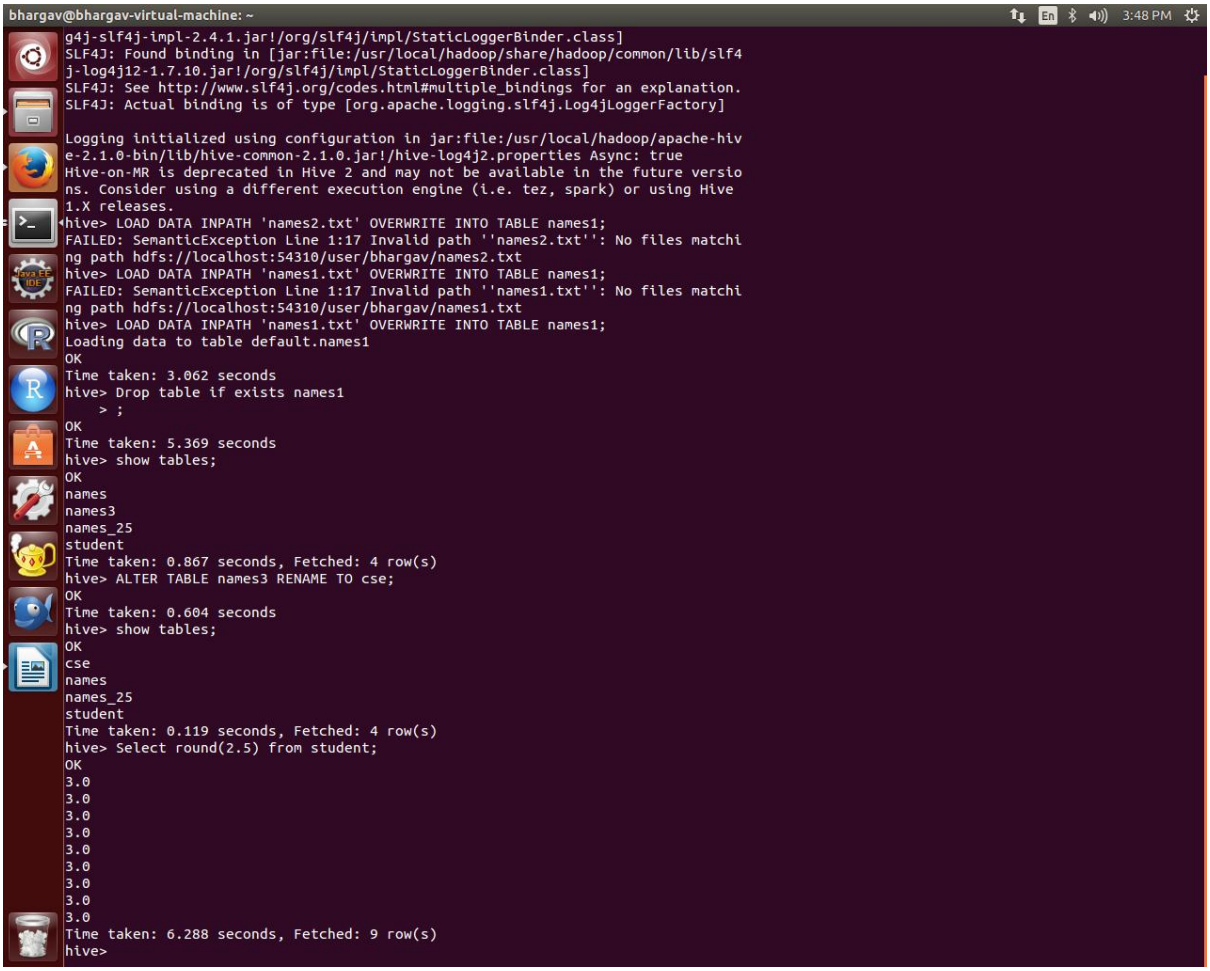
Drop table if exists names1

ALTER TABLE names1 RENAME TO names4;



Select round(2.5) from student;

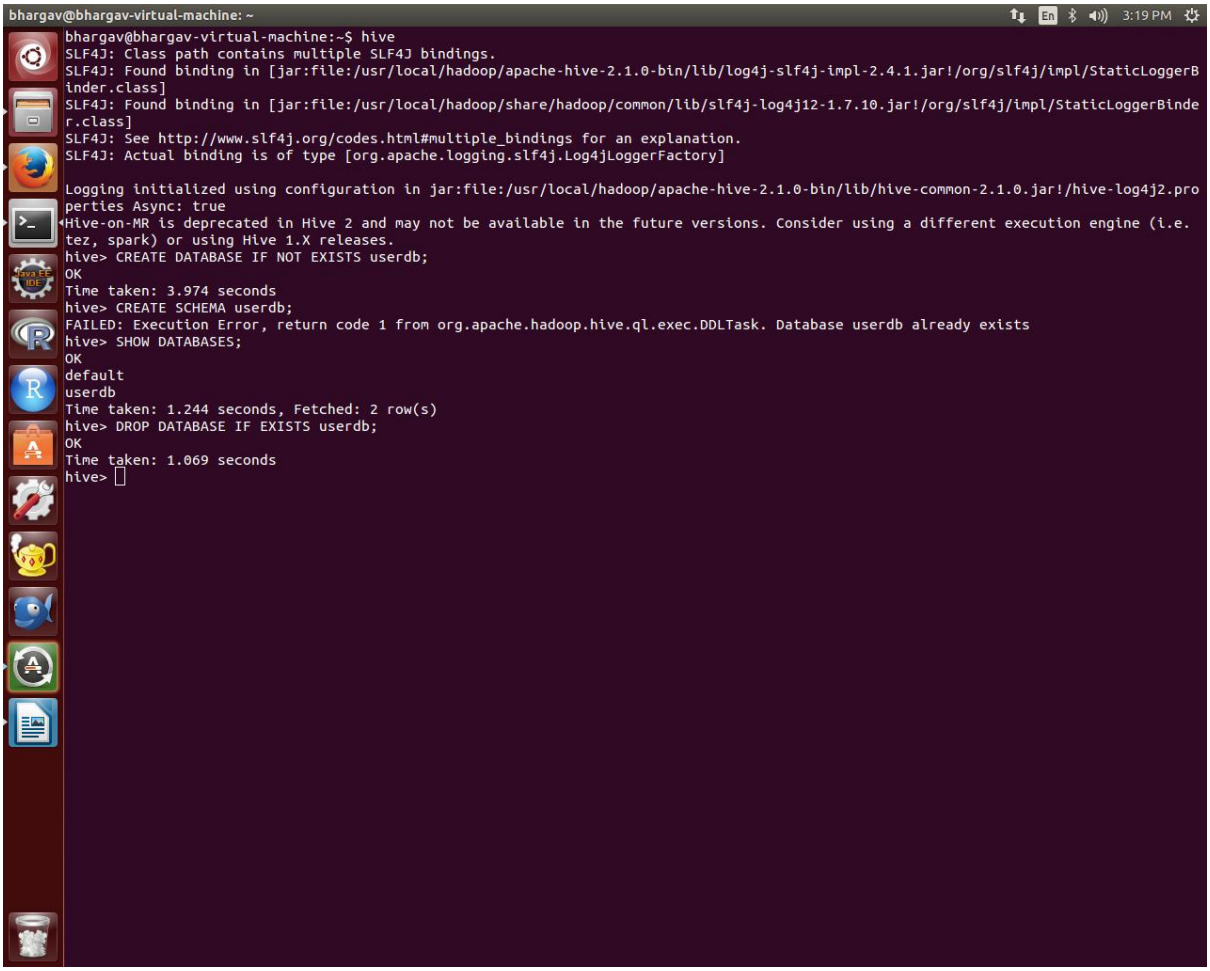
Create view names_25 where age



Drop databases

DROP DATABASE IF EXISTS userdb;

DROP DATABASE IF EXISTS userdb CASCADE; (first it drops the tables in a database)



ii)Performance techniques in partitions,bucketing

Now that you've seen it let's talk about how it works. Unlike partitioning where each value for the slicer or partition key gets its own space, in clustering a hash is taken for the field value and then distributed across buckets. In the example above, we created 10 buckets and are clustering on username. Each bucket then would contain multiple users, while restricting each username to a single bucket.

Create bucketed table over the partitioned table

Set the maximum number of reducers to the same number of buckets specified in the table metadata (i .e. 10)

```
set map.reduce.tasks = 10;
```

Use the following command

set hive.enforce.bucketing = true;
allows the correct number of reducers and the
cluster by column to be automatically selected based on the table.

```
set hive.enforce.bucketing = true ;
create table buckettxnrecsByCat(txnno INT, txndate STRING, custno INT, amount
DOUBLE, product STRING, city STRING, state STRING, spendby STRING)
partitioned by (category STRING) clustered by (state) INTO 10 buckets row format
delimited
fields terminated by ','
stored as textfile location '/user/hduser/hiveexternaldata/bucketout';
Insert into table buckettxnrecsByCat partition (category) select
txnno,txndate,custno,amount, product,city,state,spendby,category from txnrecords;
select count(1),category from buckettxnrecsByCat group by category;
SELECT txnno,product,state FROM buckettxnrecsByCat TABLESAMPLE(BUCKET
3 OUT OF 10);
```

Week 12: Migrate from Mysql database to hive using Sqoop.

Create Table in MySQL

In Cloudera VM, open the command prompt and just make sure MySQL is installed.

shell> mysql --version

mysql Ver 14.14 Distrib 5.1.66, for redhat-linux-gnu (x86_64) using readline 5.

You should always work in your own database, so create a database in MySQL using

mysql> create database sqoop;

Then:

mysql> use sqoop;

mysql> create table customer(id varchar(3), name varchar(20), age varchar(3), salary integer(10));

Query OK,

mysql> desc customer;

```
Field  | Type        | Null | Key | Default | Extra |
+--------+-------------+------+-----+---------+-------+
| id     | varchar(3)  | YES  |     | NULL    |       |
| name   | varchar(20) | YES  |     | NULL    |       |
| age    | varchar(3)  | YES  |     | NULL    |       |
 | salary | int(10)     | YES  |     | NULL    |
```

mysql> select * from customer;

Let's Start Sqooping

As you can see, the **customer** table **does not have any primary key**. I have added few records in customet table. By default, Sqoop will identify the primary key column (if present) in a table and use it as the splitting column. The low and high values for the splitting column are retrieved from the database, and the map tasks operate on evenly-sized components of the total range.

If the actual values for the primary key are not uniformly distributed across its range, then this can result in unbalanced tasks. You should explicitly choose a different column with the --split-by argument. For example, --split-by id.

Since I want to import this table directly into Hive I am adding –hive-import to my Sqoop command:

```
sqoop import --connect jdbc:mysql://localhost:3306/sqoop
--username root
-P
--split-by id
--columns id,name
--table customer
--target-dir /user/cloudera/ingest/raw/customers
--fields-terminated-by ","
--hive-import
--create-hive-table
--hive-table sqoop_workspace.customers
```

Here's what each individual Sqoop command option means:

- **connect –** Provides jdbc string
- **username** – Database username
- **-P** – Will ask for the password in console. Alternatively you can use **–password** but this is not a good practice as its visible in your job execution logs and asking for trouble. One way to deal with this is store database passwords in a file in HDFS and provide at runtime.
- **table –** Tells the computer which table you want to import from MySQL. Here, it's customer.
- **split-by –** Specifies your splitting column. I am specifying id here.
- **target-dir** – HDFS destination directory.
- **fields-terminated-by** – I have specified comma (as by default it will import data into HDFS with comma-separated values)
- **hive-import** – Import table into Hive (Uses Hive's default delimiters if none are set.)
- **create-hive-table** – Determines if set job will fail if a Hive table already exists. It works in this case.
- **hive-table** – Specifies <db_name>.<table_name>. Here it's sqoop_workspace.customers, where sqoop_workspace is my database and customers is the table name.

As you can see below, Sqoop is a map-reduce job. Notice that I am using -P for password option. While this works, but can be easliy parameterized by using –password and reading it from file.

Warning: /usr/lib/sqoop/../accumulo does not exist! Accumulo imports will fail.

Please set $ACCUMULO_HOME to the root of your Accumulo installation.

16/03/01 12:59:44 INFO sqoop.Sqoop: Running Sqoop version: 1.4.6-cdh5.5.0

Enter password:

16/03/01 12:59:54 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.

16/03/01 12:59:54 INFO tool.CodeGenTool: Beginning code generation

16/03/01 12:59:55 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `customer` AS t LIMIT 1

16/03/01 12:59:56 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `customer` AS t LIMIT 1

16/03/01 12:59:56 INFO orm.CompilationManager: HADOOP_MAPRED_HOME is /usr/lib/hadoop-mapreduce

Note: /tmp/sqoop-cloudera/compile/6471c43b5c867834458d3bf5a67eade2/customer.java uses or overrides a deprecated API.

Note: Recompile with -Xlint:deprecation for details.

16/03/01 13:00:01 INFO orm.CompilationManager: Writing jar file: /tmp/sqoop-cloudera/compile/6471c43b5c867834458d3bf5a67eade2/customer.jar

16/03/01 13:00:01 WARN manager.MySQLManager: It looks like you are importing from mysql.

16/03/01 13:00:01 WARN manager.MySQLManager: This transfer can be faster! Use the --direct

16/03/01 13:00:01 WARN manager.MySQLManager: option to exercise a MySQL-specific fast path.

16/03/01 13:00:01 INFO manager.MySQLManager: Setting zero DATETIME behavior to convertToNull (mysql)

16/03/01 13:00:01 INFO mapreduce.ImportJobBase: Beginning import of customer

16/03/01 13:00:01 INFO Configuration.deprecation: mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address

16/03/01 13:00:02 INFO Configuration.deprecation: mapred.jar is deprecated. Instead, use mapreduce.job.jar

16/03/01 13:00:04 INFO Configuration.deprecation: mapred.map.tasks is deprecated. Instead, use mapreduce.job.maps

16/03/01 13:00:05 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032

16/03/01 13:00:11 INFO db.DBInputFormat: Using read commited transaction isolation

16/03/01 13:00:11 INFO db.DataDrivenDBInputFormat: BoundingValsQuery: SELECT MIN(`id`), MAX(`id`) FROM `customer`

16/03/01 13:00:11 WARN db.TextSplitter: Generating splits for a textual index column.

16/03/01 13:00:11 WARN db.TextSplitter: If your database sorts in a case-insensitive order, this may result in a partial import or duplicate records.

16/03/01 13:00:11 WARN db.TextSplitter: You are strongly encouraged to choose an integral split column.

16/03/01 13:00:11 INFO mapreduce.JobSubmitter: number of splits:4

16/03/01 13:00:12 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1456782715090_0004

16/03/01 13:00:13 INFO impl.YarnClientImpl: Submitted application application_1456782715090_0004

16/03/01 13:00:13 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1456782715090_0004/

```
16/03/01 13:00:13 INFO mapreduce.Job: Running job:
job_1456782715090_0004
16/03/01 13:00:47 INFO mapreduce.Job: Job job_1456782715090_0004
running in uber mode : false
16/03/01 13:00:48 INFO mapreduce.Job:  map 0% reduce 0%
16/03/01 13:01:43 INFO mapreduce.Job:  map 25% reduce 0%
16/03/01 13:01:46 INFO mapreduce.Job:  map 50% reduce 0%
16/03/01 13:01:48 INFO mapreduce.Job:  map 100% reduce 0%
16/03/01 13:01:48 INFO mapreduce.Job: Job job_1456782715090_0004
completed successfully
16/03/01 13:01:48 INFO mapreduce.Job: Counters: 30
   File System Counters
      FILE: Number of bytes read=0
      FILE: Number of bytes written=548096
      FILE: Number of read operations=0
      FILE: Number of large read operations=0
      FILE: Number of write operations=0
      HDFS: Number of bytes read=409
      HDFS: Number of bytes written=77
      HDFS: Number of read operations=16
      HDFS: Number of large read operations=0
      HDFS: Number of write operations=8
   Job Counters
      Launched map tasks=4
      Other local map tasks=5
      Total time spent by all maps in occupied slots (ms)=216810
      Total time spent by all reduces in occupied slots (ms)=0
      Total time spent by all map tasks (ms)=216810
      Total vcore-seconds taken by all map tasks=216810
```

```
        Total megabyte-seconds taken by all map tasks=222013440
    Map-Reduce Framework
        Map input records=10
        Map output records=10
        Input split bytes=409
        Spilled Records=0
        Failed Shuffles=0
        Merged Map outputs=0
        GC time elapsed (ms)=2400
        CPU time spent (ms)=5200
        Physical memory (bytes) snapshot=418557952
        Virtual memory (bytes) snapshot=6027804672
        Total committed heap usage (bytes)=243007488
    File Input Format Counters
        Bytes Read=0
    File Output Format Counters
        Bytes Written=77
16/03/01 13:01:48 INFO mapreduce.ImportJobBase: Transferred 77 bytes in
104.1093 seconds (0.7396 bytes/sec)
16/03/01 13:01:48 INFO mapreduce.ImportJobBase: Retrieved 10 records.
16/03/01 13:01:49 INFO manager.SqlManager: Executing SQL statement:
SELECT t.* FROM `customer` AS t LIMIT 1
16/03/01 13:01:49 INFO hive.HiveImport: Loading uploaded data into Hive


Logging initialized using configuration in jar:file:/usr/jars/hive-common-1.1.0-
cdh5.5.0.jar!/hive-log4j.properties
OK
Time taken: 2.163 seconds
Loading data to table sqoop_workspace.customers
```

chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sqoop_workspace.db/customers/part-m-00000': User does not belong to supergroup
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sqoop_workspace.db/customers/part-m-00001': User does not belong to supergroup
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sqoop_workspace.db/customers/part-m-00002': User does not belong to supergroup
chgrp: changing ownership of 'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sqoop_workspace.db/customers/part-m-00003': User does not belong to supergroup
Table sqoop_workspace.customers stats: [numFiles=4, totalSize=77]
OK
Time taken: 1.399 seconds

Finally, let's verify the output in Hive:

hive> show databases;
OK
default
sqoop_workspace
Time taken: 0.034 seconds, Fetched: 2 row(s)
hive> use sqoop_workspace;
OK
Time taken: 0.063 seconds

hive> show tables;
OK
customers

```
Time taken: 0.036 seconds, Fetched: 1 row(s)
hive> show create table customers;
OK
CREATE TABLE `customers`(  `id` string,   `name` string)
COMMENT 'Imported by sqoop on 2016/03/01 13:01:49'
ROW FORMAT DELIMITED
  FIELDS TERMINATED BY ','
  LINES TERMINATED BY '\n'
STORED AS INPUTFORMAT
  'org.apache.hadoop.mapred.TextInputFormat'
OUTPUTFORMAT
  'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION

'hdfs://quickstart.cloudera:8020/user/hive/warehouse/sqoop_workspace.db/custo
mers'
TBLPROPERTIES (
  'COLUMN_STATS_ACCURATE'='true',
  'numFiles'='4',
  'totalSize'='77',
  'transient_lastDdlTime'='1456866115')
Time taken: 0.26 seconds, Fetched: 18 row(s)

hive> select * from customers;
OK
1    John
2    Kevin
19    Alex
3    Mark
```

4 Jenna

5 Robert

6 Zoya

7 Sam

8 George

9 Peter

Time taken: 1.123 seconds, Fetched: 10 row(s).

Experiment-1
Data Structures

1) What is data structure?

Data structures refers to the way data is organized and manipulated. It seeks to find ways to make data access more efficient. When dealing with data structure, we not only focus on one piece of data, but rather different set of data and how they can relate to one another in an organized manner.

2) What is a linked list?

A linked list is a sequence of nodes in which each node is connected to the node following it. This forms a chain-like link of data storage.

3Are linked lists considered linear or non-linear data structures?

It actually depends on where you intend to apply linked lists. If you based it on storage, a linked list is considered non-linear. On the other hand, if you based it on access strategies, then a linked list is considered linear.

4 What is the primary advantage of a linked list?

A linked list is a very ideal data structure because it can be modified easily. This means that modifying a linked list works regardless of how many elements are in the list.

5 How do you search for a target key in a linked list?

To find the target key in a linked list, you have to apply sequential search. Each node is traversed and compared with the target key, and if it is different, then it follows the link to the next node. This traversal continues until either the target key is found or if the last node is reached.

6What is the difference between Iterator and ListIterator?

Iterator traverses the elements in forward direction only whereas ListIterator traverses the elements in forward and backward direction.

No.	Iterator	ListIterator
1)	Iterator traverses the elements in forward direction only.	ListIterator traverses the elements in backward and forward directions both.
2)	Iterator can be used in List, Set and Queue.	ListIterator can be used in List only.

STACKS and QUEUES

What is the data structures used to perform recursion?

Stack. Because of its LIFO (Last In First Out) property it remembers its 'caller' so knows whom to return when the function has to return. Recursion makes use of system stack for storing the return addresses of the function calls.

Every recursive function has its equivalent iterative (non-recursive) function. Even when such equivalent iterative procedures are written, explicit stack is to be used.

Describe stack operation.

Stack is a data structure that follows Last in First out strategy.
Stack Operations:-
Push – Pushes (inserts) the element in the stack. The location is specified by the pointer.
Pop – Pulls (removes) the element out of the stack. The location is specified by the pointer
Swap: - the two top most elements of the stack can be swapped
Peek: - Returns the top element on the stack but does not remove it from the stack
Rotate:- the topmost (n) items can be moved on the stack in a rotating fashion
A stack has a fixed location in the memory. When a data element is pushed in the stack, the pointer points to the current element

Describe queue operation.

Queue is a data structure that follows First in First out strategy.
Queue Operations:
Push – Inserts the element in the queue at the end.
Pop – removes the element out of the queue from the front
Size – Returns the size of the queue
Front – Returns the first element of the queue.
Empty – to find if the queue is empty.

Discuss how to implement queue using stack.

A queue can be implemented by using 2 stacks:-
1. An element is inserted in the queue by pushing it into stack 1
2. An element is extracted from the queue by popping it from the stack 2
3. If the stack 2 is empty then all elements currently in stack 1 are transferred to stack 2 but in the reverse order
4. If the stack 2 is not empty just pop the value from stack 2.

Explain stacks and queues in detail.

A stack is a data structure based on the principle Last In First Out. Stack is container to hold nodes and has two operations - push and pop. Push operation is to add nodes into the stack and pop operation is to delete nodes from the stack and returns the top most node.
A queue is a data structure based on the principle First in First Out. The nodes are kept in an order. A node is inserted from the rear of the queue and a node is deleted from the front. The first element inserted in the first element first to delete.

SETS AND MAPS

What is the difference between List and Set ?
Set contain only unique elements while List can contain duplicate elements.
Set is unordered while List is ordered . List maintains the order in which the objects are added .
Q7 What is the difference between Map and Set ?
Map object has unique keys each containing some value, while Set contain only unique values.

Experiment-3

HDFS

1.What are the key features of HDFS?

HDFS is highly fault-tolerant, with high throughput, suitable for applications with large data sets, streaming access to file system data and can be built out of commodity hardware.

2.What is the process to change the files at arbitrary locations in HDFS?
HDFS does not support modifications at arbitrary offsets in the file or multiple writers but files are written by a single writer in append only format i.e. writes to a file in HDFS are always made at the end of the file.

3.Explain about the indexing process in HDFS
Indexing process in HDFS depends on the block size. HDFS stores the last part of the data that further points to the address where the next part of data chunk is stored.

4 Explain the difference between NAS and HDFS. NAS runs on a single machine and thus there is no probability of data redundancy whereas HDFS runs on a cluster of different machines thus there is data redundancy because of the replication protocol.
NAS stores data on a dedicated hardware whereas in HDFS all the data blocks are distributed across local drives of the machines.

In NAS data is stored independent of the computation and hence Hadoop MapReduce cannot be used for processing whereas HDFS works with Hadoop MapReduce as the computations in HDFS are moved to data.

5.What happens to a NameNode that has no data?

There does not exist any NameNode without data. If it is a NameNode then it should have some sort of data in it.

Experiment:4-6
MAPREDUCE

What are the core methods of a Reducer?

The 3 core methods of a reducer are –

1)setup () – This method of the reducer is used for configuring various parameters like the input data size, distributed cache, heap size, etc.

Function Definition- *public void setup (context)*

2)reduce () it is heart of the reducer which is called once per key with the associated reduce task.

Function Definition -public void reduce (Key,Value,context)

3)cleanup () - This method is called only once at the end of reduce task for clearing all the temporary files.

Function Definition -public void cleanup (context)

Explain about the partitioning, shuffle and sort phase Click here to Tweet

Shuffle Phase-Once the first map tasks are completed, the nodes continue to perform several other map tasks and also exchange the intermediate outputs with the reducers as required. This process of moving the intermediate outputs of map tasks to the reducer is referred to as Shuffling.

Sort Phase- Hadoop MapReduce automatically sorts the set of intermediate keys on a single node before they are given as input to the reducer.

Partitioning Phase-The process that determines which intermediate keys and value will be received by each reducer instance is referred to as partitioning. The destination partition is same for any key irrespective of the mapper instance that generated it.

Explain what is shuffling in MapReduce ?

The process by which the system performs the sort and transfers the map outputs to the reducer as inputs is known as the shuffle

Explain what is distributed Cache in MapReduce Framework ?

Distributed Cache is an important feature provided by map reduce framework. When you want to share some files across all nodes in Hadoop Cluster, DistributedCache is used. The files could be an executable jar files or simple properties file.

Explain what is JobTracker in Hadoop? What are the actions followed by Hadoop?

In Hadoop for submitting and tracking MapReduce jobs, JobTracker is used. Job tracker run on its own JVM process

Hadoop performs following actions in Hadoop

– Client application submit jobs to the job tracker

– JobTracker communicates to the Namemode to determine data location

– Near the data or with available slots JobTracker locates TaskTracker nodes

– On chosen TaskTracker Nodes, it submits the work

– When a task fails, Job tracker notify and decides what to do then.

– The TaskTracker nodes are monitored by JobTracker

Explain what combiners is and when you should use a combiner in a MapReduce Job?

To increase the efficiency of MapReduce Program, Combiners are used. The amount of data can be reduced with the help of combiner's that need to be transferred across to the reducers. If the operation performed is commutative and associative you can use your reducer code as a combiner. The execution of combiner is not guaranteed in Hadoop

Explain what is the function of MapReducer partitioner?

The function of MapReducer partitioner is to make sure that all the value of a single key goes to the same reducer, eventually which helps evenly distribution of the map output over the reducers

Explain what is difference between an Input Split and HDFS Block?

Logical division of data is known as Split while physical division of data is known as HDFS Block

Explain what happens in textinformat ?

In textinputformat, each line in the text file is a record. Value is the content of the line while Key is the byte offset of the line. For instance, Key: longWritable, Value: text

Mention what are the main configuration parameters that user need to specify to run Mapreduce Job ?

The user of Mapreduce framework needs to specify

– Job's input locations in the distributed file system

– Job's output location in the distributed file system

– Input format

– Output format

– Class containing the map function

– Class containing the reduce function

– JAR file containing the mapper, reducer and driver classes

What does combiner do?

Combiner does local aggregation of key/values pairs produced by mapper before or during shuttle and sort phase. In general, it reduces amount of data to be transferred between nodes.
The framework decides how many times to run it. Combiner may run zero, one or multiple times on the same input.

Explain mapper life cycle?

Initialization method is called before any other method is called. It has no parameters and no output.
Map method is called separately for each key/value pair. It process input key/value pairs and emits intermediate key/value pairs.

Close method runs after all input key/value have been processed. The method should close all open resources. It may also emit key/value pairs.

Explain reducer life cycle?

Initialization method is called before any other method is called. It has no parameters and no output.
Reduce method is called separately for each key/[values list] pair. It process intermediate key/value pairs and emits final key/value pairs. Its input is a key and iterator over all intermediate values associated with the same key.

Close method runs after all input key/value have been processed. The method should close all open resources. It may also emit key/value pairs.

Experiment-7

PIG

1)How is Pig Useful For?
In three categories,we can use pig .they are 1)ETL data pipline 2)Research on raw data 3)Iterative processing
Most common usecase for pig is data pipeline.Let us take one example, web based compaines gets the weblogs,so before storing data into warehouse,they do some operations on data like cleaning and aggeration operations..etc.i,e transformations on data.

2)What are the scalar datatypes in pig?

scalar datatype
int -4bytes,
float -4bytes,
double -8bytes,
long -8bytes,
chararray,
bytearray

3)What are the complex datatypes in pig?

map:

map in pig is chararray to data element mapping where element have pig data type including complex data type.
example of map ['city'#'hyd','pin'#500086]
the above example city and pin are data elements(key) mapping to values

tuple:

tuple have fixed length and it have collection datatypes.tuple containing multiple fields and also tuples are ordered.
example, (hyd,500086) which containing two fields.

bag:

A bag containing collection of tuples which are unordered,Bag constants are constructed using braces, with tuples in the bag separated by com-
mas. For example, {('hyd', 500086), ('chennai', 510071), ('bombay', 500185)}

4)Whether pig latin language is case-sensitive or not?

pig latin is some times not a case sensitive.let us see example,Load is equivalent to load.
A=load 'b' is not equivalent to a=load 'b'
UDF are also case sensitive,count is not equivalent to COUNT.

5)What is the purpose of 'dump' keyword in pig?

dump diaplay the output on the screen
dump 'processed'

6)what are relational operations in pig latin?

they are
a)for each
b)order by
c)filters
d)group
e)distinct
f)join
g)limit

Experiment-8

HIVE

1)What are the different types of tables available in HIve?
There are two types. Managed table and external table. In managed table both the data an schema in under control of hive but in external table only the schema is under control of Hive.

2)Is Hive suitable to be used for OLTP systems? Why?
No Hive does not provide insert and update at row level. So it is not suitable for OLTP system.

3)Can a table be renamed in Hive?
Alter Table table_name RENAME TO new_name

4)Can we change the data type of a column in a hive table?
Using REPLACE column option

ALTER TABLE table_name REPLACE COLUMNS ……

5)What is the default location where hive stores table data?
hdfs://namenode_server/user/hive/warehouse

6)What are the three different modes in which hive can be run?

- Local mode
- Distributed mode
- Pseudodistributed mode

7)Is there a date data type in Hive?
Yes. The TIMESTAMP data types stores date in java.sql.timestamp format

8)What are collection data types in Hive?
There are three collection data types in Hive.

- ARRAY
- MAP
- STRUCT